FPT UNIVERSITY

FACULTY OF INFORMATION TECHNOLOGY



GROUP FINAL PROJECT

FAP201

Lecturer: Phạm Thế Vinh

Class: IC2007

Group: SEMI

Students:

Trương Vũ - SE203241

Lê Anh Minh - SE201576

Nguyễn Quang Huy - SE211506

Võ Tấn Thành - SE204471

Bùi Anh Hào - SE200414

Ho Chi Minh City, Ngày 30 Tháng 10 Năm 2026

LỜI CẢM ƠN

Đầu tiên, em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến Thầy Phạm Thế Vinh trường Đại học FPT, những người đã cống hiến tâm sức cho sự nghiệp giáo dục và tận tình dìu dắt chúng em trong suốt thời gian qua.

Trong 3 năm học tập và rèn luyện tại trường Đại học FPT, em đã được chỉ bảo tận tình, được truyền đạt những kiến thức nền tảng quý báu về lĩnh vực Thiết kế Vi mạch Bán dẫn, giúp mở rộng tầm nhìn và trang bị cho em một tư duy chuyên môn vững chắc. Môi trường đại học năng động tại FPT cùng sự quan tâm từ các tổ chức đoàn thể cũng đã tạo ra nhiều sân chơi bổ ích, giúp em rèn luyện các kỹ năng mềm cần thiết. Những kiến thức và kỹ năng tích lũy được không chỉ giúp em hoàn thành tốt đồ án lần này, mà còn là tiền đề vững chắc cho các học kỳ chuyên sâu tiếp theo, cũng như hành trang để em tự tin bước vào kỳ thực tập và theo đuổi con đường Kỹ sư Thiết kế Vi mạch (Front-End RTL Design Engineer) trong tương lai.

Tiếp đến, em xin gửi lời tri ân đặc biệt đến Thầy Phạm Thế Vinh bộ môn Thiết Kế Vi Mạch, Bán Dẫn đã tạo điều kiện tối đa, cung cấp tài liệu và hướng dẫn những nền tảng thiết kế cốt lõi để em thực hiện thành công báo cáo đồ án "Thiết kế và Mô phỏng giao thức truyền thông UART-FIFO". Em vô cùng biết ơn những trải nghiệm học tập thực tế và cơ hội thực hành kỹ năng thiết kế chuyên sâu mà bộ môn đã mang lại.

Cuối cùng, em xin gửi lời cảm ơn từ tận đáy lòng đến gia đình, người thân và bạn bè - những người đã luôn là hậu phương vững chắc, là chỗ dựa tinh thần to lớn giúp em vượt qua những thử thách trong học tập để hoàn thành tốt đồ án này.

Trân trọng,

Trương Vũ, Lê Anh Minh, Nguyễn Quang Huy, Võ Tấn Thành, Bùi Anh Hào.

Thành phố Hồ Chí Minh, Ngày 1 Tháng 11 Năm 2026

SEMI

Khoa Vi Mạch, Bán Dẫn - Lớp IC2007

MỤC LỤC

TÓM TẮT BÀI BÁO CÁO....8
Chương 1. GIỚI THIỆU TỔNG QUAN ĐỀ TÀI....9
1.1. Bối cảnh và nhu cầu thực tế....9
1.2. Mục tiêu và phương pháp thực hiện đề tài....10
1.2.1. Mục tiêu của đề tài (Thiết kế IP UART có tích hợp FIFO)....10
1.2.2. Phương pháp thực hiện....10
1.3. Bảng thông số kỹ thuật của hệ thống....11
Chương 2. CƠ SỞ LÝ THUYẾT....12
2.1. Giao thức truyền thông tin nối tiếp bất đồng bộ UART....12
2.1.1. Khái niệm và đặc điểm giao tiếp Full-Duplex....12
2.1.2. Cấu trúc khung truyền vật lý (Baud rate, Start, Data và Stop bits)....13
2.2. Bài toán lệch xung nhịp (Clock Domain Crossing - CDC)....18
2.2.1. Hiện tượng bán ổn định (Metastability)....19
2.2.2. Giải pháp đồng bộ hóa bằng thanh ghi dịch (Shift Register)....20
2.3. Cơ chế bộ đệm FIFO First-Word Fall-Through (FWFT)....22
2.4. Phương pháp kiểm thử tự động với Golden Model....24
CHƯƠNG 3. THIẾT KẾ PHẦN CỨNG....27
3.1. Sơ đồ khối tổng thể của hệ thống UART....27
3.2. Thiết kế chi tiết module tạo xung nhịp (BAUD_GEN)....30
3.2.1. Nguyên lý lấy mẫu quá mức 16x (16x Oversampling)....30
3.2.2. Tính toán hệ số chia tần và sai số Baud rate....32
3.3. Thiết kế chi tiết module thu dữ liệu (UART_RX)....36
Bảng 3.8: Các hằng số suy ra của uart_rx....37
3.3.1. Sơ đồ máy trạng thái (FSM) 2 khối chuẩn mực....37
3.3.2. Thuật toán định vị điểm giữa bit (Eye-diagram center)....40
3.4. Thiết kế chi tiết module phát dữ liệu (UART_TX)....43
3.4.1. Sơ đồ máy trạng thái (FSM) và thanh ghi dịch....43
3.4.2. Xử lý khoảng nghỉ an toàn tại trạng thái IDLE....48
3.5. Thiết kế chi tiết module bộ đệm (FIFO FWFT)....50
3.5.1. Quản lý bộ đệm vòng (Circular Buffer) và cờ báo trạng thái....50
3.5.2: Giải quyết xung đột đọc/ghi đồng thời (Read-During-Write Hazard)....53
3.6. Module UART_TOP và cơ chế ghép nối vòng lặp tự động (Auto-Loopback)

UART_TOP ghép năm thực thể theo Hình 3.1 và thêm hai phép gán tổ hợp tạo nên cơ chế loopback tự động mà không cần bộ điều khiển (FSM) riêng:....55
CHƯƠNG 4. KẾT QUẢ KIỂM THỬ VÀ MÔ PHỎNG HỆ THỐNG....59
4.1. Môi trường kiểm thử Self-Checking Testbench....59
4.1.1. Khởi tạo Stimulus Generator và tích hợp Python Golden Model....60
4.1.2. Mạch giám sát tự động (Monitor) và thuật toán chấm điểm (Scoreboard)....60
4.2. Kết quả mô phỏng trên phần mềm QuestaSim....61
4.2.1. Phân tích dạng sóng kịch bản truyền đơn lẻ (Single-Byte Echo)....61
4.2.2. Phân tích dạng sóng kịch bản truyền dồn dập (Burst Stress Test)....63
4.3. Đánh giá độ tin cậy và khả năng chống nghẽn của hệ thống....64
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN CỦA ĐỀ TÀI....64

5.1. Kết luận.....64
5.2. Những khó khăn trong quá trình thiết kế.....64
5.3. Hướng phát triển trong tương lai.....65

DANH MỤC HÌNH

Hình 1.1: Sơ đồ giao tiếp UART giữa hai thiết bị..... 11
Hình 1.2: Cấu trúc của một gói thông tin trong giao thức UART..... 12
Hình 2.1: Sơ đồ khối UART_TOP..... 15
Hình 2.2: Ảnh đoạn code tham số tốc độ thay đổi trạng thái..... 16
Hình 2.3: Ảnh code tính giá trị đếm tiếp theo và tạo xung theo tick..... 16
Hình 2.4: Giản đồ xung của một khung truyền 10-bit không có Parity-bit..... 16
Hình 2.5: Ảnh code Data bit..... 18
Hình 2.6: Ảnh code Data bit tại bộ thu..... 18
Hình 2.7: Giản đồ tín hiệu xung truyền dữ liệu 8'b00000000 và đoản mạch..... 19
Hình 2.8: Đoạn code của Stop-bit tại bộ thu..... 20
Hình 2.9: Hình mô tả Clock Domain Crossing (CDC)..... 20
Hình 2.10: Hình mô tả double-flop..... 21
Hình 2.11: Hình minh họa kéo dài chu kỳ miền có clock nhanh hơn..... 21
Hình 2.12: Sử dụng module FIFO giải quyết CDC..... 21
Hình 2.13: Setup Time và Hold Time..... 22
Hình 2.14: Mạch đồng bộ hóa 2 tầng..... 23
Hình 2.15: Đoạn code Two-Stage Flip-Flop Synchronizer..... 23
Hình 2.16: Tham số cho đoạn code FSM states..... 24
Hình 2.17: Bộ đệm tiêu chuẩn FIFO..... 25
Hình 2.18: Bộ đệm FWFT FIFO..... 25
Bảng 2.1: bảng tham số bộ đệm FIFO dự án..... 26
Hình 2.19: Sơ đồ module testbench nằm trong chương trình..... 27
Hình 2.20: Ảnh minh họa Golden Model..... 27
Hình 3.1: Sơ đồ khối tổng thể UART_TOP..... 30
Hình 3.2: Sơ đồ khối bên trong của BAUD_GEN..... 34
Hình 3.3: Giản đồ thời gian của bộ đếm và xung s_tick_o..... 37
Hình 3.4: Bộ đồng bộ hóa hai flip-flop cho tín hiệu uartrx_i..... 39
Hình 3.5. Sơ đồ chuyển trạng thái của uart_rx..... 40
Hình 3.6: Điểm lấy mẫu giữa bit của uart_rx và sơ đồ mắt..... 43
Hình 3.7: Sơ đồ FSM của UART_TX..... 46
Hình 3.8: Khung truyền 8N1 của byte 0x41 trên uarttx_o (LSB first)..... 49
Hình 3.9: Giản đồ thời gian chốt chặn !txdone_o tại IDLE..... 5
Hình 3.10: Bộ đệm vòng 16 ô với wr_ptr, rd_ptr và fifo_count..... 53
Hình 3.11. So sánh đọc tổ hợp FWFT và đọc đồng bộ khi rút một phần tử..... 57
Hình 4.1: Transcript tổng kết mô phỏng: PASS 5/5, ERROR 0..... 62
Hình 4.2: Dạng sóng kiểm thử truyền đơn byte 0xA5..... 64
Hình 4.3: Dạng sóng kiểm thử truyền đơn byte 0x5A..... 64
Hình 4.4: Dạng sóng burst 0x31, 0x32, 0x33 thể hiện sự chồng lấp RX/TX..... 65
Hình 5.1. VD thực tiễn khi UART connect với CPU..... 67
Hình 5.2. VD về Parity Check trong UART..... 68
Hình 5.3. VD khi UART giao tiếp với FPGA..... 68

DANH MỤC BẢNG

Bảng 2.1: bảng tham số bộ đệm FIFO dự án..... 26
Bảng 3.1: Tham số của UART_TOP..... 30
Bảng 3.2: Các cổng giao tiếp của UART_TOP..... 31
Bảng 3.3: Vai trò của các khối trong hệ thống.....31
Bảng 3.4: Tham số và cổng của BAUD_GEN.....34
Bảng 3.5: Sai số baud tại CLK_FREQ = 50 MHz, oversampling 16..... 36
Bảng 3.6: Các thông số thời gian của hệ thống (50 MHz, 115200 baud)..... 36
Bảng 3.7: Cổng giao tiếp của uart_rx.....39
Bảng 3.9: Hoạt động của từng trạng thái trong uart_rx.....42
Bảng 3.10: Thời điểm lấy mẫu của uart_rx tính từ lúc phát hiện cạnh xuống.....44
Bảng 3.11: Cổng giao tiếp của UART_TX.....45
Bảng 3.12: Hoạt động của từng trạng thái trong UART_TX.....48
Bảng 3.13. Diễn biến thanh ghi dịch b khi phát byte 0x41 (01000001b).....49
Bảng 3.14: Diễn biến quanh thời điểm kết thúc một khung truyền..... 51
Bảng 3.15: Cổng giao tiếp của FIFO.....52.
Bảng 3.16: Quy tắc cập nhật con trỏ và bộ đếm.....53
Bảng 3.17: Ví dụ diễn biến FIFO (ADDR_WIDTH = 4)..... 54
Bảng 3.18: So sánh hai cách mô tả mảng nhớ (ước lượng cho 2 FIFO 16 × 8).. 55
Bảng 3.18: So sánh hai cách mô tả mảng nhớ.....56
Bảng 3.20: Danh sách các dây tín hiệu nội bộ của UART_TOP..... 58
Bảng 3.21: Độ trễ truyền dữ liệu qua hai tầng FIFO..... 59
Bảng 3.22: Bảng cân đối thời gian của một lượt echo.....60
Bảng 3.23: Tóm tắt các khối và kỹ thuật RTL chính.....61
Bảng 4.1: Tổng kết kết quả kiểm thử các trường hợp truyền byte..... 63

TÓM TẮT BÀI BÁO CÁO

Sự chênh lệch tốc độ giữa vi mạch xử lý và chuẩn giao tiếp UART truyền thống thường gây ra hiện tượng nghẽn dữ liệu trong các hệ thống SoC. Để giải quyết bài toán này, đồ án trình bày giải pháp thiết kế lõi IP UART song công toàn phần (Full-Duplex) tích hợp bộ đệm FIFO cơ chế First-Word Fall-Through (FWFT).

Quá trình thiết kế phần cứng được thực hiện bằng ngôn ngữ SystemVerilog, bao gồm các khối thu (RX), phát (TX), bộ đệm (FIFO) và module trung tâm tích hợp cơ chế ghép nối vòng lặp tự động (Auto-Loopback). Hệ thống được mô phỏng và xác minh khắt khe trên phần mềm QuestaSim thông qua môi trường kiểm thử tự động (Self-Checking Testbench) đối chiếu với mô hình tham chiếu (Golden Model) viết bằng Python.

Kết quả mô phỏng khẳng định mạch hoạt động ổn định với độ chính xác 100% qua các kịch bản truyền nhận dữ liệu dồn dập (Burst data). Thiết kế giải quyết triệt để vấn đề lệch miền xung nhịp (CDC) và triệt tiêu độ trễ, tạo tiền đề vững chắc để tiến hành nạp thử nghiệm xuống board mạch FPGA vật lý.

Chương 1. GIỚI THIỆU TỔNG QUAN ĐỀ TÀI

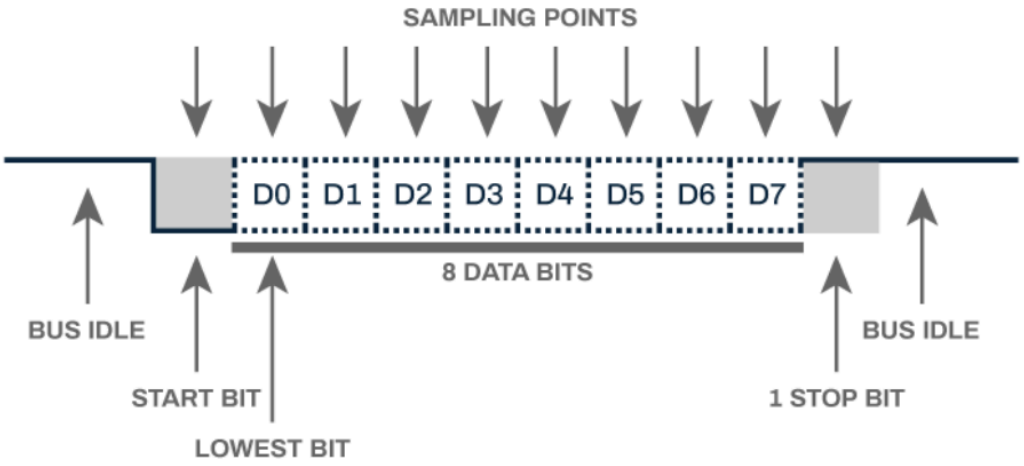
1.1. **Bối cảnh và nhu cầu thực tế**

Trong thiết kế hệ thống nhúng, vi mạch chuyên dụng (SoC) và các giải pháp vi mạch lập trình FPGA, giao thức UART (Universal Asynchronous Receiver-Transmitter) là chuẩn giao tiếp nối tiếp phổ biến nhất để giao tiếp vi điều khiển, nạp chương trình, hiển thị nhật ký gỡ lỗi (debug log) và trao đổi dữ liệu với máy tính chủ. Điểm mạnh vượt trội của UART là không cần đường xung nhịp đồng bộ (clock-less interface), giúp tiết kiệm chân I/O và tối giản hóa thiết kế phần cứng.



Hình 1.1: Sơ đồ giao tiếp UART giữa hai thiết bị.

Với đặc tính truyền thông Full-Duplex (Song công toàn phần), UART cho phép quá trình truyền (TX) và nhận (RX) dữ liệu diễn ra độc lập và song song trên hai đường vật lý riêng biệt. Tuy nhiên, tốc độ truyền nhận nối tiếp thường rất chậm so với tốc độ xử lý hàng trăm MHz của lõi xử lý bên trong. Do đó, nhu cầu thực tế đặt ra là phải thiết kế một lõi IP UART hoàn chỉnh có tích hợp cấu trúc bộ đệm kép nhằm hạn chế tối đa nguy cơ tràn dữ liệu (buffer overrun) và đảm bảo tính liên tục của luồng dữ liệu.



Hình 1.2: Cấu trúc của một gói thông tin trong giao thức UART.

1.2. **Mục tiêu và phương pháp thực hiện đề tài**

1.2.1. **Mục tiêu của đề tài (Thiết kế IP UART có tích hợp FIFO)**

- **Thiết kế hoàn chỉnh lõi IP UART:** Xây dựng các khối chức năng gồm Bộ phát (Transmitter - TX), Bộ thu (Receiver - RX) có khả năng lấy mẫu quá mức (oversampling) chống nhiễu, và Bộ tạo tốc độ Baud (Baud Rate Generator).
- **Tích hợp bộ đệm kép FIFO FWFT:** Ứng dụng kiến trúc đệm hàng đợi FIFO theo cơ chế First-Word Fall-Through (FWFT) cho cả 2 kênh RX và TX, cho phép đọc dữ liệu với độ trễ bằng 0 (Zero-latency) và tối ưu hóa tài nguyên phần cứng (LUTRAM) trên FPGA.
- **Hỗ trợ Auto-Loopback:** Tích hợp cơ chế phản hồi phần cứng tức thời phục vụ kiểm thử echo mà không cần can thiệp phần mềm bên ngoài.

1.2.2. **Phương pháp thực hiện**

- **Thiết kế RTL Front-end:** Sử dụng ngôn ngữ *SystemVerilog* tuân thủ các quy chuẩn công nghiệp, thiết kế máy trạng thái hữu hạn (FSM) 2 khối rõ ràng, triệt tiêu hoàn toàn hiện tượng sinh Latch không mong muốn và đồng bộ hóa đa tầng tín hiệu RX chống hiện tượng siêu ổn định (metastability).
- **Môi trường kiểm thử tự động:** Xây dựng Testbench tự kiểm tra kết quả (Self-Checking Testbench) kết hợp với mô hình tham chiếu bằng Python

(Python Golden Model) trên trình mô phỏng QuestaSim, đảm bảo tính đúng đắn của dữ liệu truyền nhận qua mọi tình huống biên.

Bảng dưới đây tổng hợp các thông số kỹ thuật cấu hình mặc định của hệ thống lõi IP UART, được chuẩn hóa để đảm bảo độ chính xác của tốc độ baud và sự tương thích cao trên nền tảng FPGA:

1.3. **Bảng thông số kỹ thuật của hệ thống**

Thông số (Parameter)	Giá trị mặc định	Mô tả chi tiết kỹ thuật
CLK_FREQ	50 MHz	Tần số xung nhịp hệ thống chính (Chu kỳ clock = 20 ns).
BAUD_RATE	115.200 bps	Tốc độ truyền dữ liệu nối tiếp chuẩn qua cổng UART.
DATA_WIDTH	8 bits	Độ rộng khung dữ liệu tiêu chuẩn (Khung truyền 8-N-1).
Độ sâu FIFO (ADDR_WIDTH = 16)	16 bits	Dung lượng bộ đệm FIFO độc lập cho mỗi kênh RX và kênh TX.

Chương 2. CƠ SỞ LÝ THUYẾT

2.1. Giao thức truyền thông tin nối tiếp bất đồng bộ UART

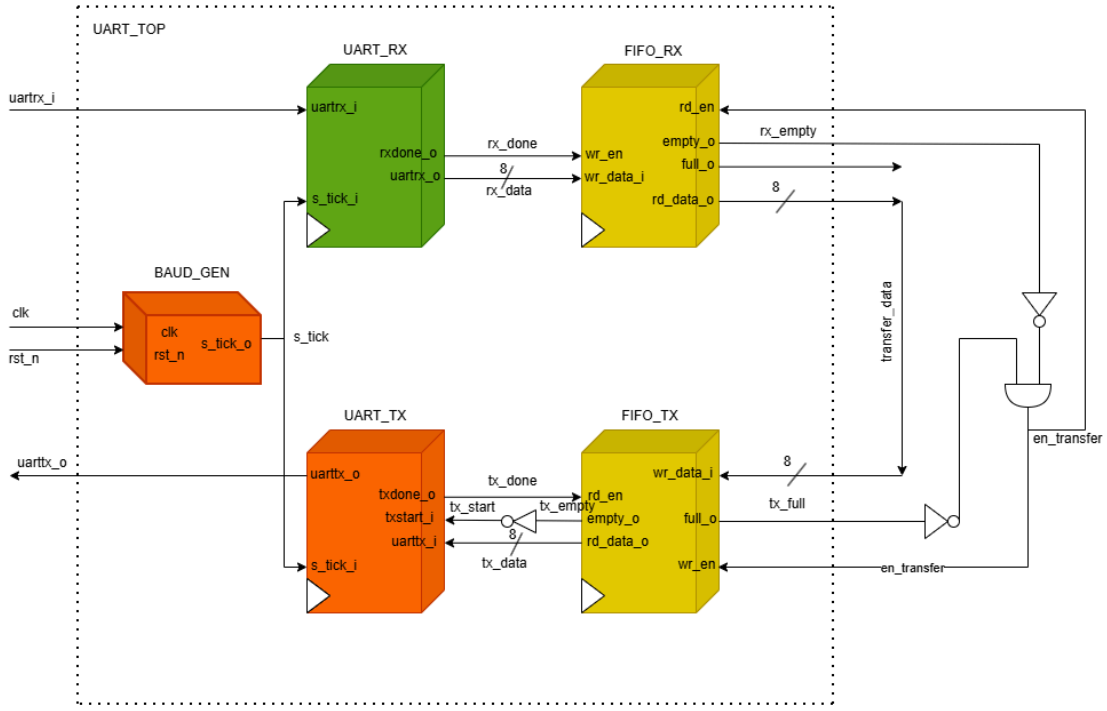
2.1.1. Khái niệm và đặc điểm giao tiếp Full-Duplex

Một trong những đặc điểm nổi bật của UART là khả năng giao tiếp Full-Duplex. Trong chế độ này, dữ liệu có thể được truyền và nhận đồng thời trên 2 đường dây dẫn riêng biệt. Nói dễ hiểu thì sẽ có 3 chế độ truyền: chỉ đọc, chỉ ghi và đọc và ghi.

TX (Transmit): Đường truyền dữ liệu từ thiết bị phát đến thiết bị nhận.

RX (Receive): Đường nhận dữ liệu từ thiết bị đối diện hay thiết bị kết nối.

Trong dự án này, hệ thống UART được thiết kế trên FPGA với kiến trúc Loopback. Dữ liệu nhận được từ cổng RX sẽ được đêm qua bộ nhớ FIFO, sau đó sẽ phản hồi lại qua cổng ra ở TX. Cơ chế này cho phép kiểm chứng toàn bộ đường truyền dữ liệu mà không cần thiết bị bên ngoài, bởi bên phát và bên nhận cùng nằm trên 1 chip FPGA duy nhất.



Hình 2.1: Sơ đồ khối UART_TOP.

Module UART_TOP trong dự án kết nối các thành phần theo chuỗi: UART_RX → FIFO_RX → Handshake Logic → FIFO_TX → UART_TX. Tín hiệu chuyển tiếp tự động en_tranfer được tạo bằng phép AND giữa tín hiệu FIFO_RX không rỗng (~rx_empty) và FIFO_TX không đầy (~tx_full), đảm bảo dữ liệu chỉ được chuyển khi cả hai bộ đệm đều sẵn sàng.

2.1.2. **Cấu trúc khung truyền vật lý (Baud rate, Start, Data và Stop bits)**

Baud rate: là số lần thay đổi trạng thái của một kênh truyền dữ liệu trong một giây , tương ứng với bit/s cho các giao thức UART.

```
module BAUD_GEN #(
    parameter CLK_FREQ    = 50_000_000, // Tần số clock hệ thống (VD: 50 MHz)
    parameter BAUD_RATE   = 115200,     // Tốc độ baud mong muốn
    parameter OVERSAMPLING = 16         // Tỷ lệ lấy mẫu (thường là 16 cho UART)
)(
    input  logic clk,
    input  logic rst_n,
    output logic s_tick_o
);
```

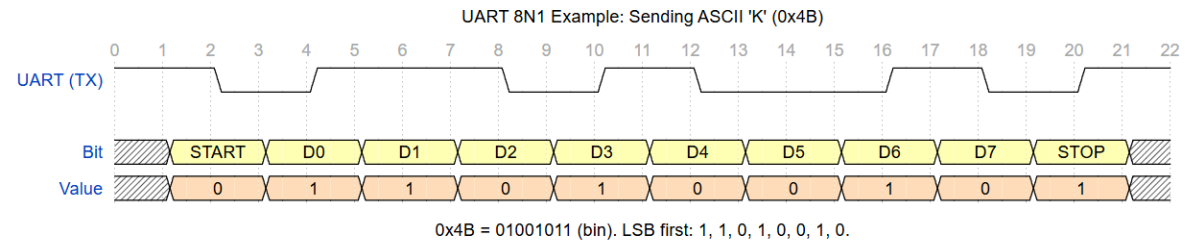
Hình 2.2: Ảnh đoạn code tham số tốc độ thay đổi trạng thái.

Đối với UART, mỗi bit được truyền trong một chu kỳ baud. Vì tần số đồng hồ hệ thống thường cao hơn rất nhiều, chúng ta cần tạo một bộ sinh baud (baud-rate generation) để tạo ra tín hiệu "tick" đánh dấu mỗi chu kỳ baud hoặc oversampling tick đánh dấu 16x mỗi chu kỳ baud để nhận giá trị chính xác hơn.

```
always_comb begin
    if (count_reg == MAX_COUNT) begin
        count_next = '0;
        s_tick_o   = 1'b1;
    end else begin
        count_next = count_reg + 1'b1;
        s_tick_o   = 1'b0;
    end
end
```

Hình 2.3: Ảnh code tính giá trị đếm tiếp theo và tạo xung theo tick.

Trong hình dưới, đường tín hiệu gồm có 3 thành phần: Start bit, Data bit và Stop bit. Tất cả 3 thành phần này cùng tạo nên một Frame truyền tín hiệu 10 bit.



Hình 2.4: Giản đồ xung của một khung truyền 10-bit không có Parity-bit.

Nguồn: UART 8N1 Explained: Frame, Baud Rate & Wiring - Node Loop

Trước khi hệ thống bắt đầu truyền tín hiệu, đường truyền UART luôn ở mức 1 được gọi là Idle Frame. Bắt cách kéo tín hiệu lên mức cao (bit 1), hệ thống truyền đang thông báo hiện tại hệ thống đang trống, không có frame nào được truyền đi. Đồng thời việc Idle luôn ở mức cao giúp hệ thống phát hiện có lỗi phần cứng hay bị đứt dây, bên

cạnh đó tạo nên tính liên tục giữa các frame truyền. Trong ảnh 2.1 với tín hiệu kết thúc (Stop-bit) luôn được kéo lên mức cao sau mỗi frame truyền, việc kéo tín hiệu idle lên mức cao giúp hệ thống không cần phải chuyển tín hiệu từ 1 về 0 nhằm tạo tính hiệu liên tục cho khung truyền tiếp theo.

Start-bit: là tín hiệu bắt buộc phải có trong một frame truyền, start-bit thông báo cho hệ thống biết có một frame dữ liệu đang được truyền đi. Bằng cách, hạ tín hiệu điện áp về 0v (Low) hệ thống sẽ bắt được tín hiệu start, tín hiệu Idle của UART luôn được cố định ở mức cao (Active High) khi có tín hiệu kéo điện áp xuống mực 0v trong đúng một chu kỳ đóng vai trò như một xung kích hoạt để báo cho bộ thu chuẩn bị nhận dữ liệu và đồng bộ pha đếm nhịp. Vậy làm thế nào để phân biệt được khi bị đoản mạch tín hiệu truyền của UART vẫn sẽ kéo về mức 0V ? Trên thực tế việc phát hiện là tín hiệu đoản mạch và tín hiệu start có thể dựa vào tín hiệu **stop** ở cuối mỗi frame sẽ được nói đến sau. Vì tín hiệu tín hiệu stop-bit luôn ở trạng thái mức cao nên khi có tín hiệu đoản mạch mạch sẽ không được kéo lên điện áp mức cao lúc này hệ thống sẽ phát hiện đây không phải là tín hiệu hợp lệ.

Data-bit: là tín hiệu chứa dữ liệu cần truyền chính của mỗi frame, đối với dự án lần này độ rộng của dữ liệu là 8-bit (từ bit D0 đến bit D7). Thứ tự truyền của các bit theo chuẩn LSB First (Least Significant Bit First), ngay sau tín hiệu start Bit D0 được truyền đi ngay lần lượt theo sau là D2, D3 cho đến D7. Tại bộ phát UART_TX, thanh ghi dịch sẽ dịch phải mỗi chu kì bit: **n_b = {1'b0, b[DATA_WIDTH - 1: 1]}** và phát ra **n_tx = b[0].**

```
DATA: begin
  n_tx = b[0];
  if (s_tick_i) begin
    if (scount == 4'd15) begin
      n_scount = 4'd0;
      n_b = {1'b0, b[DATA_WIDTH-1 : 1]};
      if (bcount == DATA_WIDTH - 1) begin
        n_state = STOP;
      end else begin
        n_bcount = bcount + 4'b1;
      end
    end else begin
      n_scount = scount + 4'b1;
    end
  end
end
```

Hình 2.5: Ảnh code Data bit.

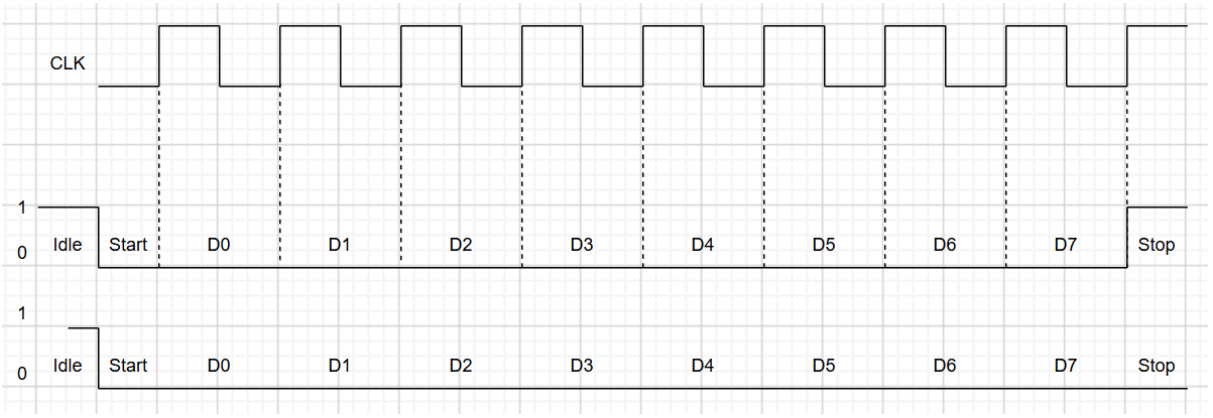
Tại bộ thu: Chốt tuần tự từng bit vào vị trí tương ứng: data_reg[bit_count] <= rx_sync.

```
DATA: begin
    if (s_tick_i) begin
        if (tick_count == TICKS_PER_BIT - 1) begin
            tick_count <= 0;
            data_reg[bit_count] <= rx_sync;
```

Hình 2.6: Ảnh code Data bit tại bộ thu.

Stop-bit: là bit tín hiệu thông báo đã kết thúc một frame truyền tín hiệu cho khối nhận biết và thiết lập lại trạng thái nghỉ (Idle). Bằng cách đưa logic của bit dữ liệu cuối cùng D7 về tín hiệu 1'b1 (Active-High). Nhằm tạo tính liên tục cho trạng thái nghỉ của frame tiếp theo, đồng thời việc kéo mức logic của tín hiệu stop về mức cao như đã đề cập trong phần start bit để phân biệt hiện tượng đoản mạch. Khi xuất hiện đoản mạch, đường truyền sẽ mất điện tín hiệu về mức thấp (Active-Low) để phân biệt

với việc truyền một gói tin đặc biệt có dữ liệu như sau 8'b00000000. Chúng ta sẽ dựa vào tín hiệu stop, bằng việc luôn để tín hiệu stop ở mức cao khối tiếp nhận sẽ biết đây là tín hiệu truyền gói dữ liệu 8'b00000000.



Hình 2.7: Giản đồ tín hiệu xung truyền dữ liệu 8'b00000000 và đoản mạch.

Khi xuất hiện hiện tượng đoản mạch điều dễ thấy nhất ở tín hiệu stop khi có đoạn mạch không còn kéo lên mức cao được nữa mặc dù đã hết một frame truyền tín hiệu. Điều này dễ dàng phát hiện thông qua biến đếm chu kỳ truyền và biến nhận tín hiệu trong khối thu để phát hiện đường truyền có bị đoản mạch hay không.

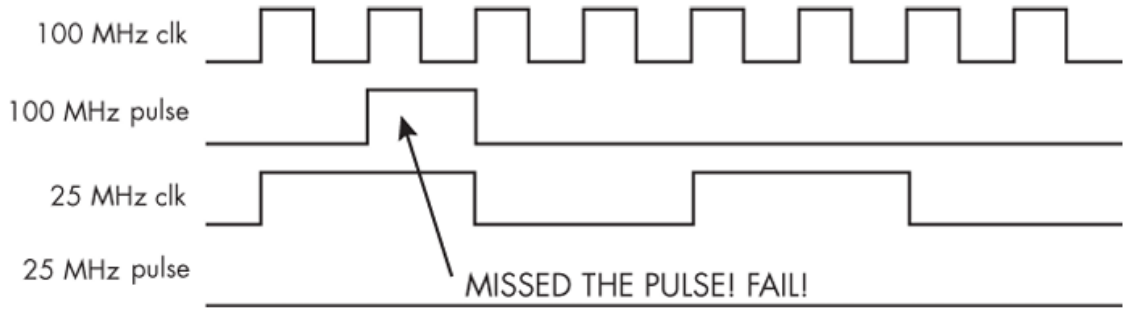
```
STOP: begin
    if (s_tick_i) begin
        if (tick_count == SB_TICK) begin
            tick_count <= 0;
            state <= IDLE;

            if (rx_sync == 1'b1) begin
                uartrx_o  <= data_reg;
                rxdone_o <= 1'b1;
            end
        end else begin
            tick_count <= tick_count + 1'b1;
        end
    end
end
```

Hình 2.8: Đoạn code của Stop-bit tại bộ thu.

2.2. **Bài toán lệch xung nhịp (Clock Domain Crossing - CDC)**

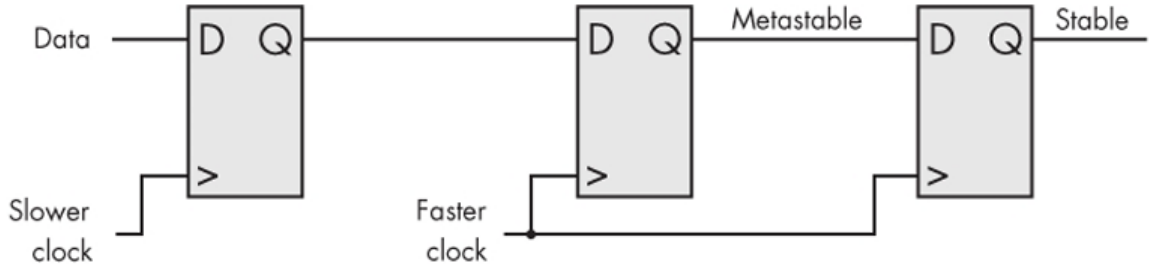
Trong giao thức UART, tín hiệu dữ liệu đến từ bên ngoài (uartrx_i) là một tín hiệu bất đồng bộ (asynchronous) so với xung nhịp nội bộ của FPGA. Tín hiệu này có thể thay đổi trạng thái tại bất kỳ thời điểm nào, không phụ thuộc vào cạnh lên hay cạnh xuống của đồng hồ hệ thống. Đây chính là bài toán Clock Domain Crossing (CDC) – bài toán lệch miền xung nhịp, một trong những thách thức quan trọng nhất trong thiết kế số.



Hình 2.9: Hình mô tả Clock Domain Crossing (CDC).

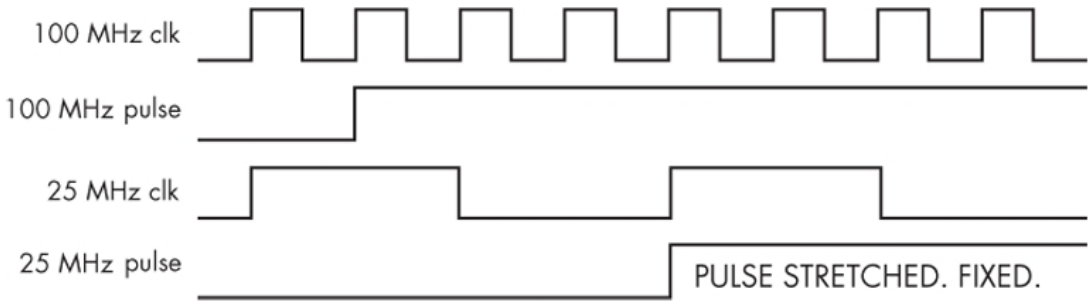
Để khắc phục được tình trạng không bắt được tín hiệu khi truyền dữ liệu ở 2 miền có xung khác nhau, người ta đã đề ra các giải pháp cho các tình huống cụ thể khác nhau như:

Từ Nơi có clock chậm hơn về nơi có clock nhanh hơn: sử dụng thêm một con flip-flop nữa được gọi là cấu trúc double-flop.



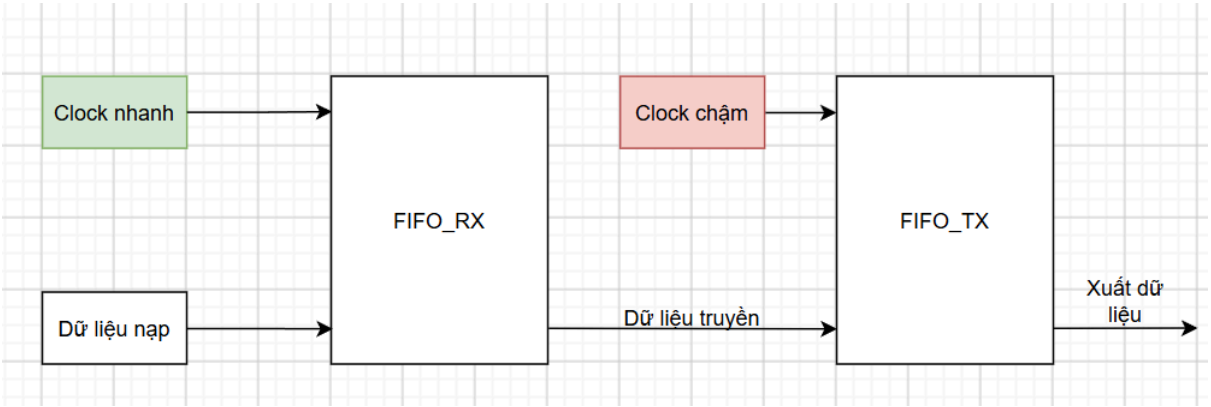
Hình 2.10: Hình mô tả double-flop.

Từ nơi có clock nhanh hơn về nơi có clock chậm hơn: đây là một trường hợp phức tạp hơn của bài toán lệch miền xung. Cách giải quyết bài toán này đó là kéo dài chu kỳ của clock nhanh hơn dài thêm ít nhất bằng 2 lần chu kì của clock chậm hơn.



Hình 2.11: Hình minh họa kéo dài chu kỳ miền có clock nhanh hơn.

Một hướng khác để khắc phục tình trạng không bắt được tín hiệu khi truyền dữ liệu từ nơi có clock nhanh về nơi có clock chậm là sử dụng module FIFO. Clock truyền nhanh hơn sẽ được nối vào khối FIFO_RX và clock nhận tín hiệu chậm hơn được nối vào khối FIFO_TX. Tất cả dữ liệu truyền sẽ được nạp vào khối FIFO_RX khi muốn đọc dữ liệu ra khối FIFO_TX sẽ yêu cầu khối FIFO_RX truyền tín hiệu từ từ theo clock nhận tín hiệu mà không sợ không bắt được tín hiệu.

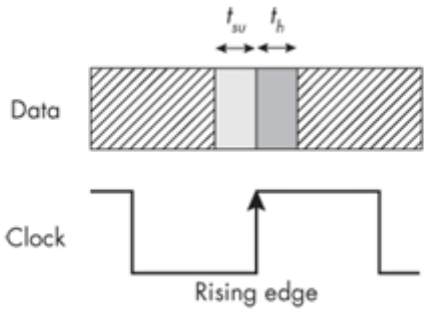


Hình 2.12: Sử dụng module FIFO giải quyết CDC.

2.2.1. Hiện tượng bán ổn định (Metastability)

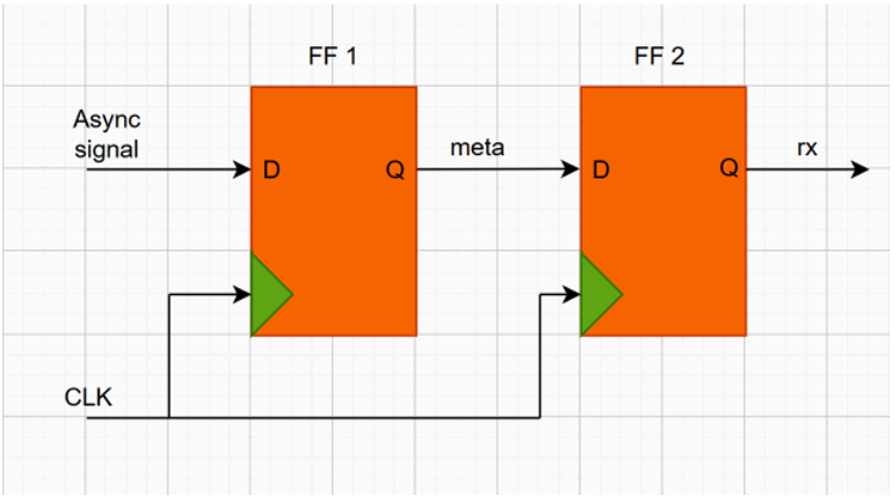
Vì có sự khác nhau giữa các giao thức bên ngoài và bên trong, từ đó trong trong FPGA lại sinh ra một bài toán mới mang tên "lệch miền xung nhịp" - Clock Domain Crossing một vấn đề khá phổ biến trong quá trình học tập về

FPGA. Chính từ bài toán lệch miền xung nhịp từ đó sinh ra một hiện tượng. Trạng thái siêu ổn định, hiện tượng này xuất hiện khi một đường truyền tín hiệu đi từ một module có xung nhịp này được truyền sang một module có xung nhịp khác hoàn toàn module cũ. Tín hiệu bị rời vào vùng **Setup Time** hoặc **Hold Time** của Flip-Flop đầu tiên lấy mẫu, Flip-Flop này sẽ rời vào trạng thái siêu ổn định (Metastable) điện áp ngõ ra lơ lửng giữa 2 mức tín hiệu 0 và 1 và phải mất một khoảng thời gian không xác định để tín hiệu ổn điện về 2 mức tín hiệu 0 hoặc 1, dẫn đến lỗi logic cho FSM.



Hình 2.13: Setup Time và Hold Time.

2.2.2. Giải pháp đồng bộ hóa bằng thanh ghi dịch (Shift Register)



Hình 2.14: Mạch đồng bộ hóa 2 tầng.

Để tránh tình trạng siêu ổn định dự án đã áp dụng giải pháp hệ thống đã sử dụng giải pháp mạch đồng bộ hóa 2 tầng Flip-Flop (Two-Stage Flip-Flop Synchronizer).

```
//================================================
// 1. Synchronize asynchronous UART input
//================================================
reg rx_meta;
reg rx_sync;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        rx_meta <= 1'b1;
        rx_sync <= 1'b1;
    end else begin
        rx_meta <= uartrx_i;
        rx_sync <= rx_meta;
    end
end
```

Hình 2.15: Đoạn code Two-Stage Flip-Flop Synchronizer.

Với **rx_meta:** bắt tín hiệu bất đồng bộ uartrx_i. Nếu xảy ra vi phạm Timing, thời gian một chu kì xung clock với f = 50MHz → T = 20ns đủ dài để điện áp của ngõ ra của rx_meta giải phóng trạng thái siêu ổn định và tự hội tụ về mức điện áp mang tín hiệu 0 hoặc 1 trước sườn lên của xung clock kế tiếp. Tầng 2 **rx_sync:** chốt lại giá trị đã ổn định từ rx_meta, cung cấp tín hiệu hoàn toàn động bộ với xung nhịp hệ thống cho máy trạng thái FSM xử lý.

```
//
// 2. FSM states
//================================================
localparam [1:0] IDLE  = 2'b00;
localparam [1:0] START = 2'b01;
localparam [1:0] DATA  = 2'b10;
localparam [1:0] STOP  = 2'b11;

reg [1:0] state;

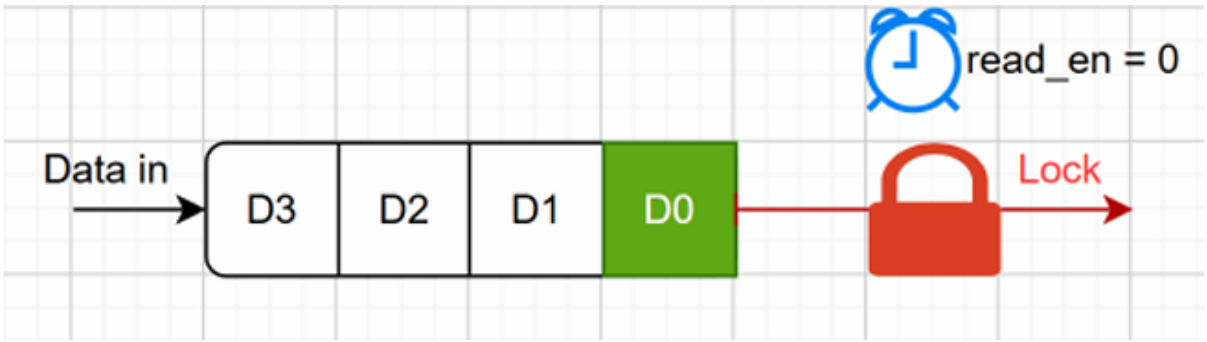
// Counter widths depend on configuration
localparam TICK_COUNT_WIDTH =
    (TICKS_PER_BIT <= 2) ? 1 : $clog2(TICKS_PER_BIT);

localparam BIT_COUNT_WIDTH =
    (DATA_WIDTH <= 2) ? 1 : $clog2(DATA_WIDTH);

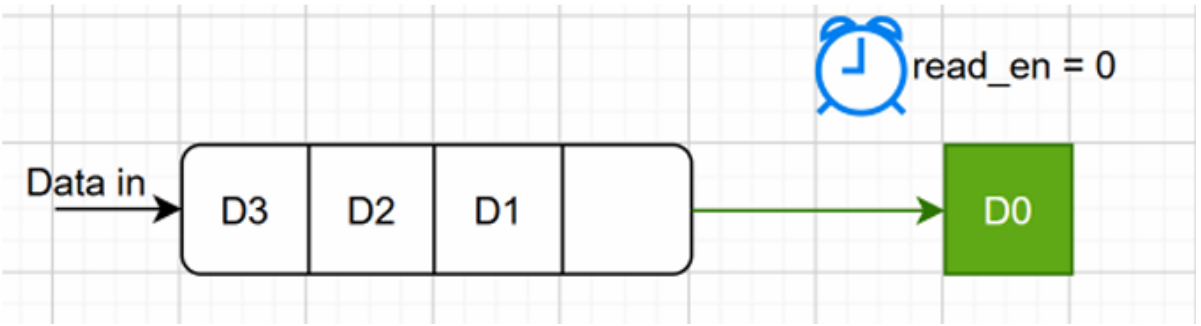
reg [TICK_COUNT_WIDTH-1:0] tick_count;
reg [BIT_COUNT_WIDTH-1:0]  bit_count;
reg [DATA_WIDTH-1:0]       data_reg;
```

Hình 2.16: Tham số cho đoạn code FSM states.

2.3. **Cơ chế bộ đệm FIFO First-Word Fall-Through (FWFT)**



Hình 2.17: Bộ đệm tiêu chuẩn FIFO.



Hình 2.18: Bộ đệm FWFT FIFO.

FIFO (First-In First-Out) là một cấu trúc dữ liệu hàng đợi, trong đó dữ liệu được ghi vào trước sẽ được đọc ra trước. Trong thiết kế UART, bộ đệm FIFO đóng vai trò then chốt trong việc giải quyết sự chênh lệch tốc độ giữa bên phát và bên nhận, đặc biệt trong trường hợp truyền dồn dập (burst transmission) khi nhiều byte được gửi liên tiếp.

Dự án sử dụng kiến trúc FIFO với cơ chế First-Word Fall-Through (FWFT). Đặc điểm chính của FWFT khác biệt so với FIFO truyền thống ở chỗ:

- FIFO truyền thống: Khi muốn đọc dữ liệu, bên đọc phải phát tín hiệu rd_en = 1, sau đó đợi một hoặc nhiều chu kỳ clock để dữ liệu xuất hiện ở ngõ ra rd_data_o. Điều này tạo ra độ trễ đọc (read latency).
- FIFO FWFT: Dữ liệu đầu tiên trong hàng đợi luôn "trồi sẵn" (fall-through) ở ngõ ra rd_data_o ngay khi FIFO không rỗng, mà không cần phát tín hiệu rd_en. Tín hiệu rd_en chỉ có tác dụng xác nhận rằng dữ liệu đã được đọc xong và cần chuyển sang phần tử tiếp theo.

Trong module FIFO của dự án, cơ chế FWFT được hiện thực bằng một câu lệnh gán tổ hợp đơn giản:

assign rd_data_o = fifo_mem[rd_ptr];

Câu lệnh trên cho thấy rd_data_o luôn trỏ đến phần tử tại vị trí con trỏ đọc (rd_ptr) trong mảng nhớ fifo_mem, bất kể tín hiệu rd_en có được kích hoạt hay không. Khi rd_en = 1 và FIFO không rỗng, con trỏ đọc rd_ptr tăng lên 1, làm rd_data_o tự động trỏ đến phần tử tiếp theo.

Tham số	Giá trị	Mô tả
DATA_WIDTH	8 bit	Độ rỗng dữ liệu mỗi ô nhớ
ADDR_WIDTH	4 bit	Số bit địa chỉ: 16 ô
Dung lượng	16 byte	16*8 = 128 bit = 16 byte
Cơ chế đọc	FWFT	Dữ liệu luôn sẵn ngõ ra

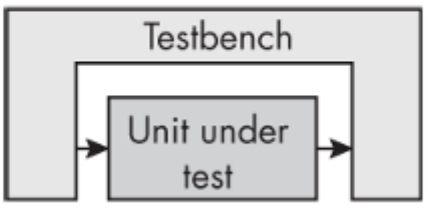
Bảng 2.1: bảng tham số bộ đệm FIFO dự án.

Hệ thống UART trong dự án sử dụng hai bộ đệm FIFO: FIFO_RX (đệm dữ liệu nhận từ UART_RX) và FIFO_TX (đệm dữ liệu chuẩn bị phát qua UART_TX). Mạch Handshake tự động chuyển dữ liệu từ FIFO_RX sang FIFO_TX thông qua tín hiệu en_transfer = (~rx_empty) & (~tx_full). Cơ chế FWFT giúp đảm bảo rằng ngay khi dữ liệu có sẵn trong FIFO_TX, module UART_TX có thể bắt đầu phát ngay lập tức mà không mất thêm chu kỳ clock chờ đọc.

Bộ đếm fifo_count theo dõi số lượng phần tử hiện có trong FIFO. Trạng thái đầy (full_o) được xác nhận khi fifo_count bằng dung lượng tối đa (2^ADDR_WIDTH = 16), và trạng thái rỗng (empty_o) khi fifo_count bằng 0. Logic cập nhật bộ đếm xử lý đồng thời ba trường hợp: chỉ ghi (tăng 1), chỉ đọc (giảm 1), và ghi/đọc đồng thời (giữ nguyên).

2.4. **Phương pháp kiểm thử tự động với Golden Model**

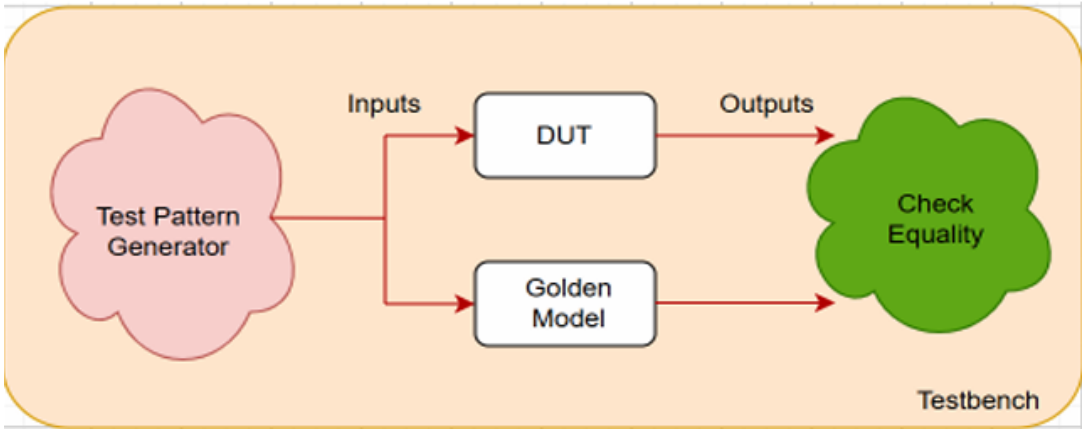
Testbench cho chương trình: là một module được tạo ra nhằm mô phỏng, kiểm tra và xác minh tính đúng đắn của một khối thiết kế phần cứng trước khi được nạp lên chip FPGA. Module testbench được quan sát dưới dạng sóng vuông.



Hình 2.19: Sơ đồ module testbench nằm trong chương trình.

Nguồn: Getting Started with FPGAs

Trong quy trình thiết kế phần cứng số, việc kiểm chứng (verification) đóng vai trò quan trọng không kém so với việc thiết kế. Một trong những phương pháp kiểm chứng hiệu quả và có tính hệ thống cao là sử dụng Golden Model – một mô hình tham chiếu chuẩn được xây dựng bằng ngôn ngữ bậc cao (ví dụ Python, C++) để mô phỏng chính xác hành vi mong muốn của phần cứng.



Hình 2.20: Ảnh minh họa Golden Model.

Phương pháp kiểm thử với Golden Model trong dự án được thực hiện qua ba bước chính:

Bước 1 - Xây dựng Golden Model bằng Python: File GOLDEN_MODEL.py trong dự án mô hình hóa toàn bộ hệ thống UART_TOP bằng các lớp Python. Lớp FifoGolden mô phỏng hành vi của module FIFO (bao gồm cơ chế FWFT), lớp UartTopGolden mô phỏng đường dữ liệu UART_RX → FIFO_RX → Handshake → FIFO_TX → UART_TX. Mỗi phương thức trong Golden Model tương ứng 1:1 với một khối phần cứng:

- uart_rx_process(): Mô phỏng thanh ghi dịch của UART_RX, nhận chuỗi 10 bit (1 Start + 8 Data LSB-first + 1 Stop) và dịch từng bit vào thanh ghi.
- handshake_process(): Mô phỏng mạch chuyển tiếp tự động, di chuyển dữ liệu từ FIFO_RX sang FIFO_TX khi điều kiện en_transfer = (~rx_empty) & (~tx_full) được thỏa mãn.

- uart_tx_process(): Mô phỏng bộ phát UART_TX, đóng gói dữ liệu thành khung 10 bit và xuất ra đầu ra.

Bước 2 – Sinh kịch bản kiểm thử và đáp án chuẩn: Golden Model thực thi các kịch bản kiểm thử giống hệt testbench SystemVerilog:

- Test 1: Gửi byte 0xA5 (1010_0101) – kiểm tra mẫu bit xen kẽ.
- Test 2: Gửi byte 0x5A (0101_1010) – kiểm tra mẫu bit đảo ngược.
- Test 3: Gửi burst 3 bytes liên tiếp (0x31, 0x32, 0x33) – kiểm tra khả năng đệm FIFO.

Sau khi chạy, Golden Model xuất các giá trị kỳ vọng (expected values) ra file golden_vectors.hex. File này chứa danh sách các byte dưới dạng hexadecimal, mỗi byte trên một dòng:

```
a5
5a
31
32
33
```

Bước 3 – So sánh tự động trong Testbench: File UART_TOP_TB.sv nạp file golden_vectors.hex bằng lệnh $readmemh và lưu vào mảng expected_mem. Trong quá trình mô phỏng, testbench:

- Gửi dữ liệu: Task send_byte() tạo khung UART 10 bit (Start + 8 Data + Stop) và đặt lên tín hiệu uartrx_i của DUT.
- Giám sát đầu ra: Khối Monitor chờ cạnh xuống trên uarttx_o (Start bit), đợi 0.5 chu kỳ bit để đến giữa Start bit, sau đó lấy mẫu 8 bit data theo thứ tự LSB-first.
- So sánh tự động: Mỗi byte nhận được từ DUT (rx_data_out) được so sánh với giá trị tương ứng trong expected_mem[test_idx]. Kết quả PASS hoặc FAIL được in ra kèm theo thông tin chi tiết (thời gian, giá trị kỳ vọng, giá trị thực tế).

- Tổng kết: Cuối mô phỏng, testbench in tổng số PASS và ERROR, đưa ra kết luận UART_TOP TEST PASSED hoặc FAILED.

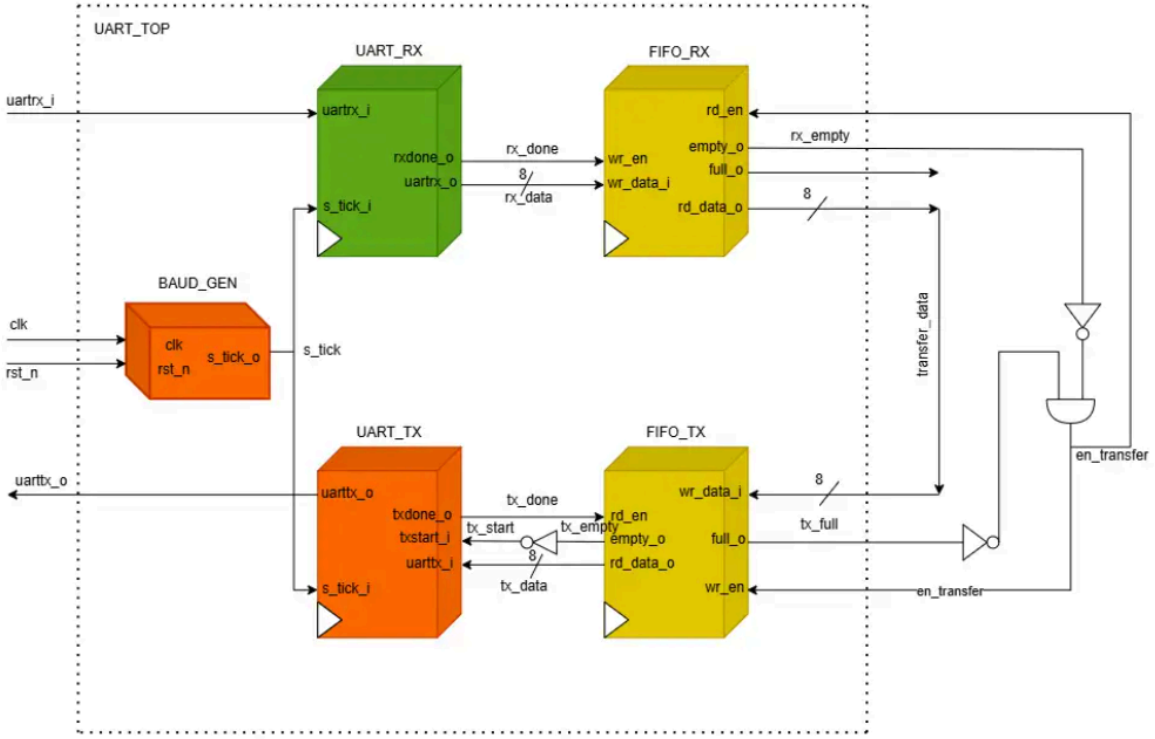
Phương pháp Golden Model mang lại nhiều ưu điểm:

- Tính tự động: Không cần kiểm tra thủ công trên waveform, toàn bộ quá trình so sánh được thực hiện tự động bởi testbench.
- Tính mở rộng: Có thể dễ dàng thêm các kịch bản kiểm thử mới bằng cách bổ sung test case trong Python và chạy lại Golden Model.
- Tính nhất quán: Đảm bảo rằng kết quả phần cứng (RTL simulation) luôn khớp với kết quả phần mềm (Python model), giúp phát hiện sớm các lỗi thiết kế.
- Tính kiểm chứng được: File golden_vectors.hex đóng vai trò như một "đáp án chuẩn" có thể được tái sử dụng và kiểm tra bởi bất kỳ ai mà không cần hiểu chi tiết mã nguồn.

CHƯƠNG 3. THIẾT KẾ PHẦN CỨNG

3.1. Sơ đồ khối tổng thể của hệ thống UART

Hệ thống được ghép trong UART_TOP gồm khối BAUD_GEN tạo xung lấy mẫu, khối UART_RX thu dữ liệu nối tiếp, FIFO_RX lưu byte nhận được, FIFO_TX lưu byte chờ phát và khối UART_TX phát dữ liệu ra đường nối tiếp. Hai FIFO được nối với nhau bằng tín hiệu en_transfer để thực hiện cơ chế loopback tự động RX → FIFO_RX → FIFO_TX → TX.



Hình 3.1: Sơ đồ khối tổng thể UART_TOP

Tham số	Giá trị	Ý nghĩa
DATA_WIDTH	8	Số bit dữ liệu của một khung UART, đồng thời là bề rộng dữ liệu của FIFO.
ADDR_WIDTH	4	Số bit địa chỉ của FIFO; độ sâu mỗi FIFO là 2^ADDR_WIDTH = 16 ô nhớ.
BAUD_RATE	115200	Tốc độ baud mong muốn (bit/s).
CLK_FREQ	50_000_000	Tần số xung nhịp hệ thống (Hz).

Bảng 3.1: Tham số của UART_TOP

Tên tín hiệu	Hướng	Độ rộng (bit)	Chức năng
clk	input	1	Xung nhịp hệ thống 50 MHz, tích cực cạnh lên; cấp cho cả 5 thực thể.
rst_n	input	1	Reset bất đồng bộ, tích cực mức thấp. Đưa các FSM về IDLE, uarttx_o về 1, xóa bộ đếm baud, con trỏ và bộ đếm của FIFO.

uartrx_i	input	1	Đường nhận dữ liệu nối tiếp, mức nghỉ bằng 1; bất đồng bộ với clk nên được đồng bộ hóa trong uart_rx.
uarttx_o	output	1	Đường phát dữ liệu nối tiếp, mức nghỉ bằng 1; là đầu ra thanh ghi của UART_TX nên không có glitch.

Bảng 3.2: Các cổng giao tiếp của UART_TOP

Thực thể	Module	Vai trò
u_baud_gen	BAUD_GEN	Chia tần clk để tạo xung s_tick có tần số bằng 16 lần tốc độ baud.
u_uart_rx	UART_RX	Đồng bộ hóa uartrx_i, lấy mẫu giữa bit, ghép 8 bit thành 1 byte song song, kiểm tra bit STOP.
u_fifo_rx	FIFO (FWFT)	Đệm các byte nhận được từ UART_RX.
u_fifo_tx	FIFO (FWFT)	Đệm các byte chờ phát; tách tốc độ giữa phía thu và phía phát.
u_uart_tx	UART_TX	Nạp byte từ FIFO_TX và phát khung 8N1 ra uarttx_o.

Bảng 3.3: Vai trò của các khối trong hệ thống

Nguyên lý hoạt động tổng thể

Hệ thống hoạt động theo luồng dữ liệu một chiều RX → FIFO → FIFO → TX, được đồng bộ bởi xung s_tick do khối BAUD_GEN cấp:

1) **Tạo xung lấy mẫu:** u_baud_gen đếm xung clk 50 MHz và cứ 27 chu kỳ phát một xung s_tick rộng 1 chu kỳ clk (tần số 16 × baud). Xung này được cấp đồng thời cho u_uart_rx và u_uart_tx làm tín hiệu cho phép, nên hai khối chạy cùng tốc độ baud.

2) **Thu dữ liệu:** u_uart_rx đồng bộ hóa uartrx_i, phát hiện bit START, lấy mẫu từng bit ở giữa chu kỳ bit; khi đủ một khung hợp lệ, nó phát xung rx_done kèm byte rx_data để ghi vào u_fifo_rx.

3) **Chuyển dữ liệu:** Khi u_fifo_rx không rỗng và u_fifo_tx chưa đầy (en_transfer = 1), trong cùng một chu kỳ clock byte ở đầu u_fifo_rx (transfer_data) được ghi vào u_fifo_tx và đồng thời được rút khỏi u_fifo_rx.

4) **Phát dữ liệu:** Khi u_fifo_tx có dữ liệu (tx_start = ~tx_empty), u_uart_tx nạp byte tx_data và phát khung 8N1 ra uarttx_o, mỗi bit kéo dài 16 xung s_tick.

5) **Giải phóng FIFO_TX:** Kết thúc khung, tx_done rút byte vừa phát khỏi u_fifo_tx, chuẩn bị cho byte kế tiếp.

Các khối giao tiếp với nhau chỉ bằng cờ empty/full và các xung done, không cần tín hiệu yêu cầu/xác nhận riêng; FIFO tách tốc độ giữa phía thu và phía phát.

3.2. Thiết kế chi tiết module tạo xung nhịp (BAUD_GEN)

3.2.1. Nguyên lý lấy mẫu quá mức 16x (16x Oversampling)

UART là giao tiếp bất đồng bộ: bên thu không nhận được xung clock từ bên phát mà phải tự xác định thời điểm đọc từng bit dựa trên tốc độ baud đã thỏa thuận trước. Nếu bên thu chỉ lấy mẫu một lần mỗi bit (1×), thời điểm bắt đầu bit START chỉ được biết với độ chính xác cả một chu kỳ bit, nên điểm lấy mẫu có thể rơi sát cạnh chuyển mức, nơi tín hiệu kém ổn định nhất.
Phương pháp lấy mẫu quá mức 16 lần chia mỗi chu kỳ bit thành 16 khoảng nhỏ (tick). Khi đó:

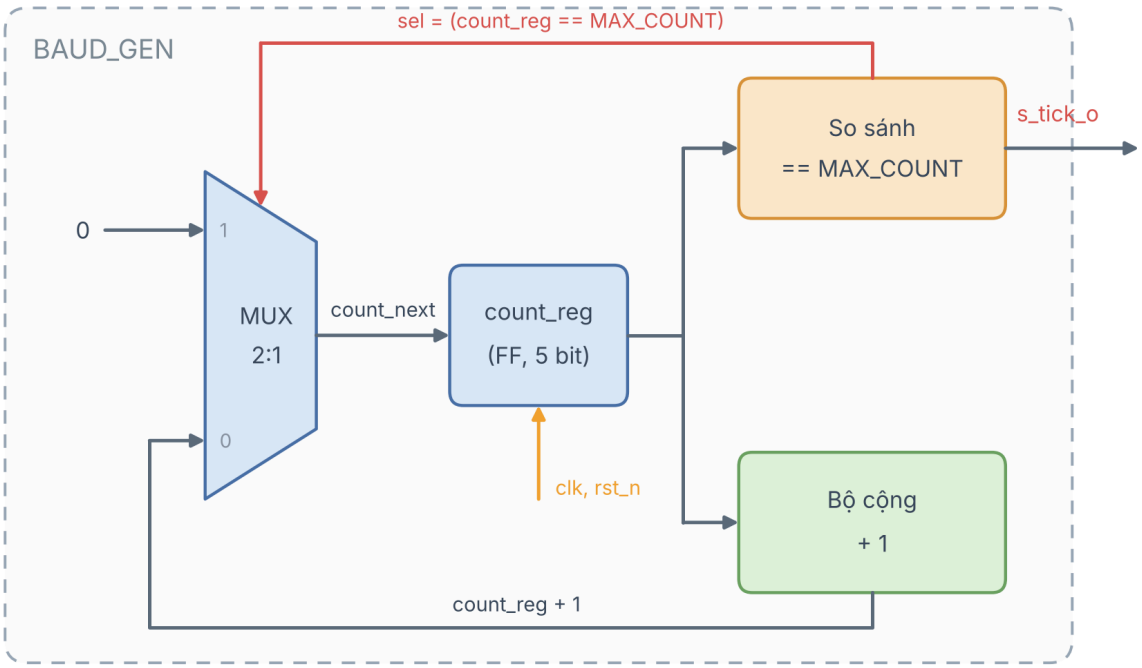
- Cạnh xuống của bit START được phát hiện với độ phân giải 1/16 chu kỳ bit (6,25 %) thay vì cả một bit.
- Bộ thu đếm 8 tick để tới **giữa** bit START rồi cứ 16 tick lấy mẫu một lần, nên luôn đọc ở giữa chu kỳ bit, xa nhất hai cạnh chuyển mức (xem mục 3.3.2).
- Có thể loại bỏ xung nhiễu hẹp: nếu tại giữa bit START tín hiệu đã trở về 1 thì đó chỉ là nhiễu, bộ thu bỏ qua.
- Mỗi bit của khung truyền có độ dài đúng 16 tick nên cả UART_RX lẫn UART_TX chỉ cần các bộ đếm nhỏ 4 bit.

BAUD_GEN do đó tạo ra xung s_tick_o có tần số bằng 16 × BAUD_RATE = 1,8432 MHz (với 115200 baud). Xung này được dùng làm tín hiệu cho phép (Clock Enable)

cho các FSM phía sau, còn các flip-flop vẫn chạy trên cùng xung clk 50 MHz.

Tên	Hướng / loại	Độ rộng	Chức năng
CLK_FREQ	parameter	-	Tần số clock hệ thống (mặc định 50 MHz).
BAUD_RATE	parameter	-	Tốc độ baud mong muốn (mặc định 115200).
OVERSAMPLING	parameter	-	Hệ số lấy mẫu quá mức, mặc định 16.
clk	input	1	Xung nhịp hệ thống.
rst_n	input	1	Reset bất đồng bộ mức thấp, xóa bộ đếm về 0.
s_tick_o	output	1	Xung lấy mẫu tích cực mức cao, rộng đúng 1 chu kỳ clk.

Bảng 3.4: Tham số và cổng của BAUD_GEN



Hình 3.2: Sơ đồ khối bên trong của BAUD_GEN

Nguyên lý hoạt động của BAUD_GEN: Khối chỉ gồm một thanh ghi đếm, một bộ cộng, một bộ so sánh và một bộ mux:

1) Thanh ghi count_reg (5 bit) tăng thêm 1 sau mỗi chu kỳ clk nhờ bộ cộng + 1.

2) Bộ so sánh kiểm tra count_reg == MAX_COUNT (= 26). Khi điều kiện đúng, s_tick_o lên 1 trong đúng chu kỳ đó.

3) Cũng khi đó tín hiệu sel chọn đầu vào 0 của mux nên count_next = 0; bộ đếm quay về 0 ở cạnh clock kế tiếp và s_tick_o hạ xuống.

4) Khi rst_n = 0, count_reg được xóa về 0 bất đồng bộ nên xung đầu tiên luôn xuất hiện sau đúng 27 chu kỳ kể từ lúc hết reset.

Dãy giá trị của count_reg lặp lại 0, 1, 2, …, 25, 26, 0, 1, … (27 trạng thái), tức một xung s_tick_o sau mỗi 27 chu kỳ clk (Hình 3.3).

3.2.2. Tính toán hệ số chia tần và sai số Baud rate

Số chu kỳ clk giữa hai xung s_tick liên tiếp (hệ số chia) được tính theo công thức:

$$BAUDTICK = \left\lfloor \frac{CLK_FREQ}{BAUD_RATE \times 16} \right\rfloor$$

Trong mã nguồn, bộ đếm đếm từ 0 đến MAX_COUNT (phép chia nguyên của Verilog tự động lấy phần nguyên), nên chu kỳ đếm đúng bằng BAUDTICK xung clk:

$$MAX_COUNT = BAUDTICK - 1$$

Bộ đếm phải biểu diễn được MAX_COUNT + 1 = BAUDTICK giá trị khác nhau (từ 0 đến MAX_COUNT), nên số bit tối thiểu là logarit cơ số 2 làm tròn lên:

$$COUNTER_WIDTH = \lceil \log_2(MAX_COUNT + 1) \rceil = \lceil \log_2 BAUDTICK \rceil$$

Với CLK_FREQ = 50 MHz và BAUD_RATE = 115200:

- 50 000 000 / (115200 × 16) = 27,127 → BAUDTICK = 27; MAX_COUNT = 26; COUNTER_WIDTH = 5 bit.
- Chu kỳ s_tick = 27 × 20 ns = 540 ns; thời gian 1 bit = 16 × 540 ns = 8,64 µs (lý thuyết: 1/115200 = 8,68 µs).

Do BAUDTICK phải là số nguyên nên tốc độ baud thực tế khác tốc độ mong muốn. Công thức tính tốc độ baud thực tế và sai số tương đối:

$$Baud_{thực\ tế} = \frac{CLK_FREQ}{BAUDTICK \times 16}$$

$$Sai\ số\ (\%) = \frac{Baud_{thực\ tế} - BAUD_RATE}{BAUD_RATE} \times 100$$

Với cấu hình mặc định: Baud thực tế = 50 000 000 / (27 × 16) = 115 740,74 baud, do đó sai số = (115 740,74 − 115 200) / 115 200 × 100 ≈ **+0,469 %**. Vì phép chia bị cắt phần thập phân nên baud thực tế luôn nhanh hơn một chút so với lý thuyết.

BAUD_RATE	50M / (16×BAUD)	BAUDTICK (làm tròn xuống)	Baud thực tế	Sai số
9600	325,52	325	9 615,38	+0,160 %
19200	162,76	162	19 290,12	+0,469 %
38400	81,38	81	38 580,25	+0,469 %
57600	54,25	54	57 870,37	+0,469 %

115200	**27,13**	**27**	**115 740,74**	**+0,469 %**
230400	13,56	13	240 384,62	+4,334 % (vượt ngưỡng)

Bảng 3.5: Sai số baud tại CLK_FREQ = 50 MHz, oversampling 16

Ngưỡng ±2 %: Trong một khung 10 bit, bộ thu lấy mẫu bit STOP ở thời điểm 9,5 chu kỳ bit tính từ cạnh xuống của bit START. Nếu tốc độ phát và thu lệch nhau một lượng ε thì điểm lấy mẫu bị trôi khoảng 9,5 × ε chu kỳ bit; điểm lấy mẫu phải còn nằm trong bit đang xét (độ trôi < 0,5 bit), nên giới hạn lý thuyết khoảng ±5 %. Thực hành lấy ngưỡng an toàn ±2 % cho **tổng** sai số của cả hai đầu truyền. Với thiết kế này:

- Sai số chia của BAUD_GEN là +0,469 %, nhỏ hơn 2 %, chừa khoảng 1,5 % cho thiết bị đối tác và sai số dao động thạch anh (thường chỉ cỡ 0,005 %).
- Tại điểm lấy mẫu bit STOP, độ trôi tích lũy chỉ khoảng 9,5 × 0,469 % ≈ 4,5 % chu kỳ bit, rất nhỏ so với giới hạn 50 %.
- Từ 230400 baud trở lên, hệ số chia quá nhỏ làm sai số vượt 2 % (Bảng 3.5); khi đó cần tăng CLK_FREQ hoặc dùng bộ chia phân số.

Đại lượng	Công thức	Giá trị
Chu kỳ clock	1 / CLK_FREQ	20 ns
Chu kỳ xung s_tick	BAUDTICK × 20 ns	27 × 20 ns = 540 ns
Tần số s_tick	CLK_FREQ / BAUDTICK	1,852 MHz
Thời gian 1 bit	16 × 540 ns	8,64 µs
Thời gian 1 khung (10 bit)	160 × 540 ns	86,4 µs (4320 chu kỳ clk)
Nửa bit (START → giữa bit)	8 × 540 ns	4,32 µs
Tốc độ dữ liệu hữu ích	115 741 baud / 10 bit	≈ 11,57 KB/s

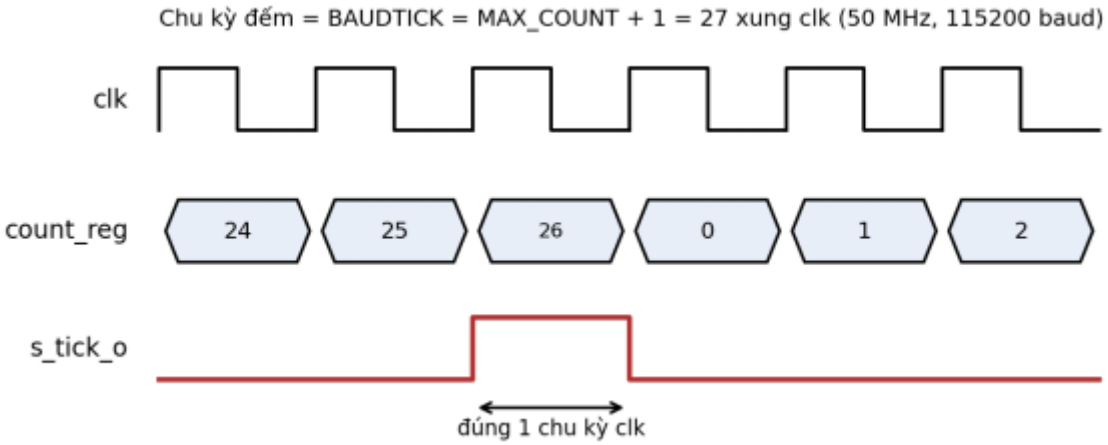
Bảng 3.6: Các thông số thời gian của hệ thống (50 MHz, 115200 baud)

Thiết kế RTL. Bộ đếm tuần hoàn được mô tả theo kiểu hai khối: khối tuần tự always_ff chỉ cập nhật count_reg, khối tổ hợp always_comb tính count_next và xung s_tick_o:

```
localparam int MAX_COUNT = CLK_FREQ / (BAUD_RATE * OVERSAMPLING) - 1;
localparam int COUNTER_WIDTH = $clog2(MAX_COUNT + 1);
logic [COUNTER_WIDTH-1:0] count_reg, count_next;

always_ff @(posedge clk or negedge rst_n) begin
   if (!rst_n) count_reg <= '0;
   else        count_reg <= count_next;
end

always_comb begin
   if (count_reg == MAX_COUNT) begin
      count_next = '0;              // quay vòng về 0
      s_tick_o   = 1'b1;            // tạo xung tick
   end else begin
      count_next = count_reg + 1'b1;
      s_tick_o   = 1'b0;
   end
end
```



Hình 3.3: Giản đồ thời gian của bộ đếm và xung s_tick_o

- **Xung rộng đúng 1 chu kỳ clk**: s_tick_o chỉ bằng 1 khi count_reg == MAX_COUNT; ở cạnh clock kế tiếp bộ đếm quay về 0 nên điều kiện không còn đúng, không cần mạch tách xung phụ.

- **Dùng làm Clock Enable, không làm clock**: mọi flip-flop vẫn chạy trên cùng clk, nên không phát sinh miền clock mới, không cần xử lý CDC và việc phân tích thời gian (STA) đơn giản.

- **Đầu ra dạng Moore**: s_tick_o được giải mã từ thanh ghi count_reg, nên thay đổi ổn định sau cạnh clock; MAX_COUNT, COUNTER_WIDTH là hằng số tính lúc biên dịch (bộ đếm chỉ rộng 5 bit).

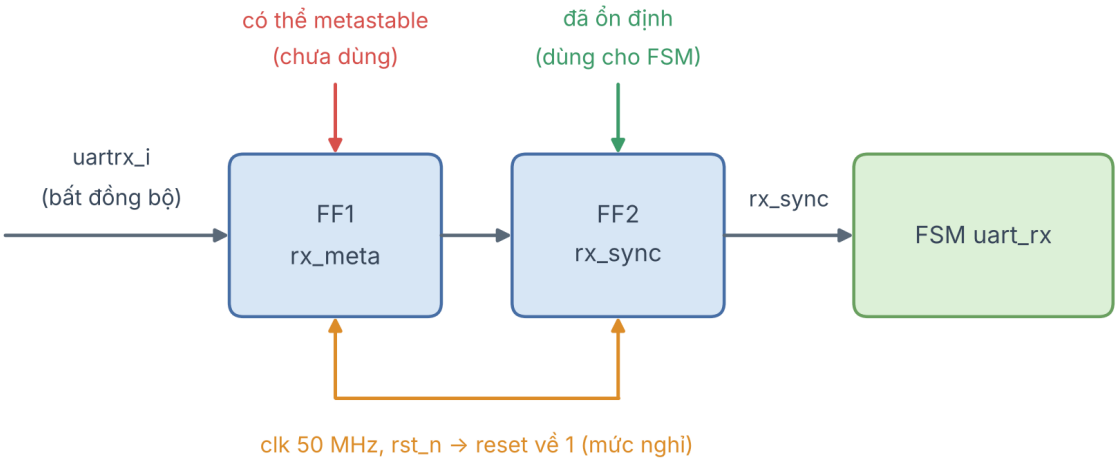
3.3. Thiết kế chi tiết module thu dữ liệu (UART_RX)

Module uart_rx chuyển luồng bit nối tiếp trên uartrx_i thành một byte song song. Các tham số: DATA_WIDTH = 8, TICKS_PER_BIT = 16, HALF_BIT_TICK = TICKS_PER_BIT / 2 = 8 và SB_TICK = TICKS_PER_BIT − 1 = 15.

Tên tín hiệu	Hướng	Độ rộng (bit)	Chức năng
clk	input	1	Xung nhịp hệ thống.
rst_n	input	1	Reset bất đồng bộ mức thấp.
uartrx_i	input	1	Đường dữ liệu nối tiếp (bất đồng bộ), mức nghỉ bằng 1.
s_tick_i	input	1	Xung lấy mẫu từ BAUD_GEN, dùng làm Clock Enable cho FSM.
rxdone_o	output	1	Xung báo UART_RX đã nhận hoàn tất một byte, nối đến `wr_en` của FIFO_RX.
uartrx_o	output	8 bits	Bus dữ liệu 8-bit chứa byte nhận được, nối đến `wr_data_i` của FIFO_RX.

Bảng 3.7: Cổng giao tiếp của uart_rx

Đồng bộ hóa tín hiệu vào: Chân uartrx_i đến từ bên ngoài, không đồng bộ với clk, nên nếu đưa trực tiếp vào FSM có thể gây hiện tượng siêu ổn định (metastability). Tín hiệu được cho qua hai flip-flop nối tiếp rx_meta → rx_sync (cả hai reset về 1, tức mức nghỉ của đường truyền); toàn bộ FSM chỉ dùng rx_sync. Độ trễ 2 chu kỳ clock (40 ns) rất nhỏ so với 1 tick (540 ns) nên không ảnh hưởng đến độ chính xác lấy mẫu.



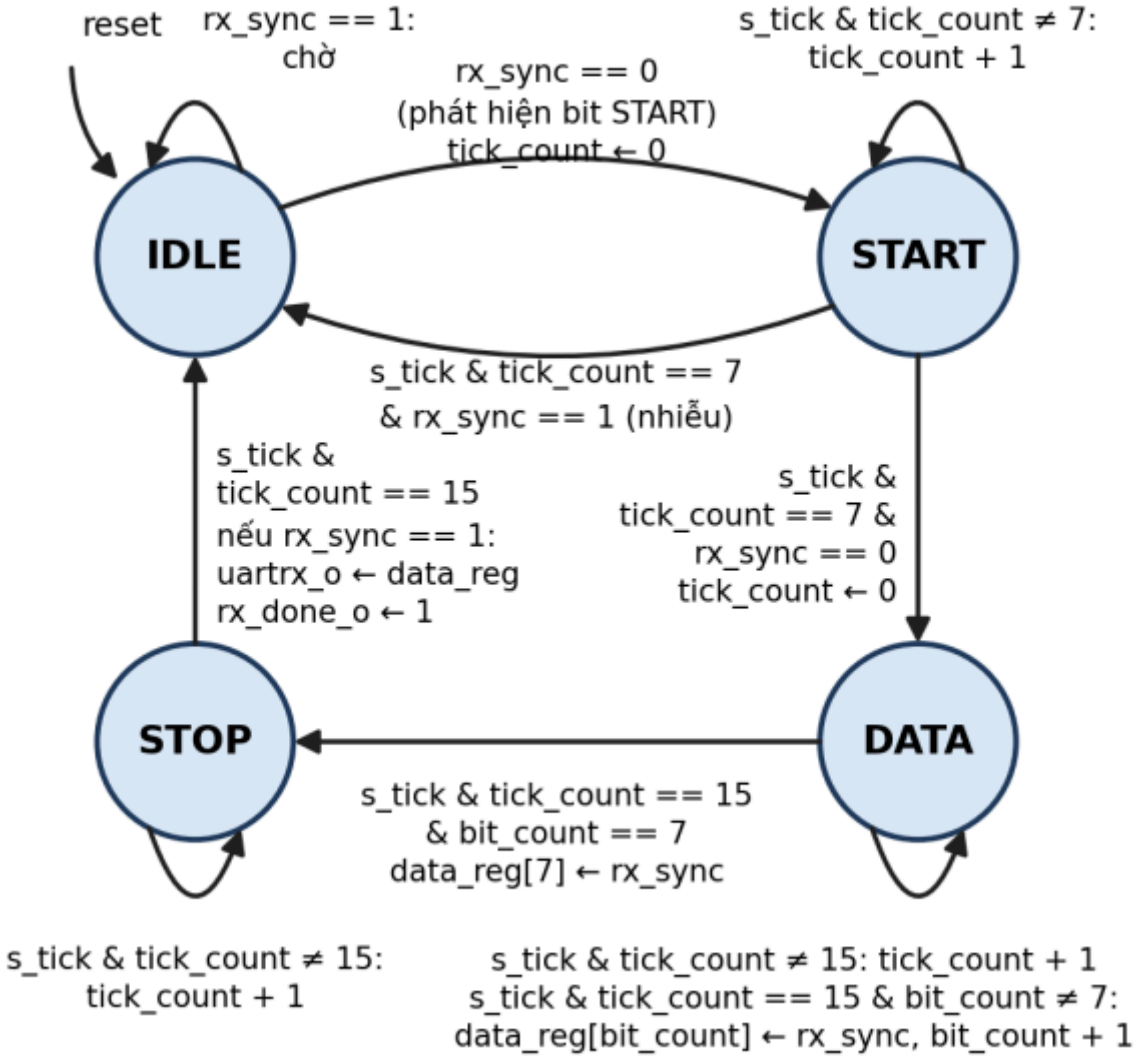
Hình 3.4: Bộ đồng bộ hóa hai flip-flop cho tín hiệu uartrx_i

Hằng số	Công thức	Giá trị (mặc định)
TICKS_PER_BIT	tham số	16
HALF_BIT_TICK	TICKS_PER_BIT / 2	8
SB_TICK	TICKS_PER_BIT − 1	15
TICK_COUNT_WIDTH	$clog2(TICKS_PER_BIT)	4 bit (đếm 0–15)
BIT_COUNT_WIDTH	$clog2(DATA_WIDTH)	3 bit (đếm 0–7)
Mã hóa trạng thái	IDLE=00, START=01, DATA=10, STOP=11	2 bit

Bảng 3.8: Các hằng số suy ra của uart_rx

3.3.1. Sơ đồ máy trạng thái (FSM) 2 khối chuẩn mực

FSM có 4 trạng thái mã hóa 2 bit: IDLE, START, DATA, STOP, cùng các thanh ghi tick_count (4 bit, đếm tick), bit_count (3 bit, đếm số bit đã nhận) và data_reg (8 bit, chứa dữ liệu đang ghép). Mô hình tách hai phần: phần **trạng thái kế tiếp** (logic tổ hợp xác định điều kiện chuyển trạng thái và giá trị kế tiếp của các bộ đếm) và phần **thanh ghi** (cập nhật tại cạnh lên clk, reset bất đồng bộ về IDLE). Trong mã nguồn uart_rx, hai phần này được viết gộp trong một khối always @(posedge clk or negedge rst_n) với phép gán không chặn (<=), nên sau tổng hợp cho cùng một cấu trúc: các thanh ghi state, tick_count, bit_count, data_reg cùng mạng logic tổ hợp phía trước chúng.



Hình 3.5. Sơ đồ chuyển trạng thái của uart_rx

Trạng thái	Trường hợp	Điều kiện	Hành động	Trạng thái kế
IDLE	1. Đường truyền rảnh	rx_sync == 1	Giữ tick_count, bit_count bằng 0; tiếp tục chờ.	IDLE
IDLE	2. Phát hiện bit START	rx_sync == 0	Đặt tick_count <= 0, bắt đầu đếm tick.	START
START	1. Chưa tới giữa bit	s_tick_i & tick_count != 7	tick_count <= tick_count + 1.	START
START	2. Tới giữa bit START	s_tick_i & tick_count == 7	Xóa tick_count; kiểm tra lại rx_sync: - bằng 0: đúng là bit START - bằng 1: xung nhiễu, bỏ qua	DATA (nếu rx_sync == 0) IDLE (nếu rx_sync == 1)
DATA	1. Chưa tới giữa bit	s_tick_i & tick_count != 15	tick_count <= tick_count + 1.	DATA
DATA	2. Tới giữa bit dữ liệu	s_tick_i & tick_count == 15	Xóa tick_count; lấy mẫu data_reg[bit_count] <= rx_sync: - bit_count != 7: tăng bit_count - bit_count == 7: xóa bit_count	DATA (bit chưa đủ) STOP (đủ 8 bit)
STOP	1. Chưa tới giữa bit	s_tick_i & tick_count != 15	tick_count <= tick_count + 1.	STOP
STOP	2. Tới giữa bit STOP	s_tick_i & tick_count == 15	Xóa tick_count; kiểm tra rx_sync: - bằng 1: rx_data_o <= data_reg, rx_done_o <= 1 - bằng 0: lỗi khung, bỏ byte	IDLE

Bảng 3.9: Hoạt động của từng trạng thái trong uart_rx

Ghi chú: ở START, DATA, STOP, khi s_tick_i = 0 *mọi thanh ghi giữ nguyên giá trị; chỉ khi có xung* s_tick_i *mới xét hai trường hợp nêu trên.*

Một số điểm thiết kế đáng chú ý:

- Kiểm tra bit STOP (framing check): chỉ khi rx_sync == 1 ở giữa bit STOP, byte mới được chép sang rx_data_o và rx_done_o mới được kích. Khung có bit STOP sai bị loại bỏ, không đưa dữ liệu rác vào FIFO.

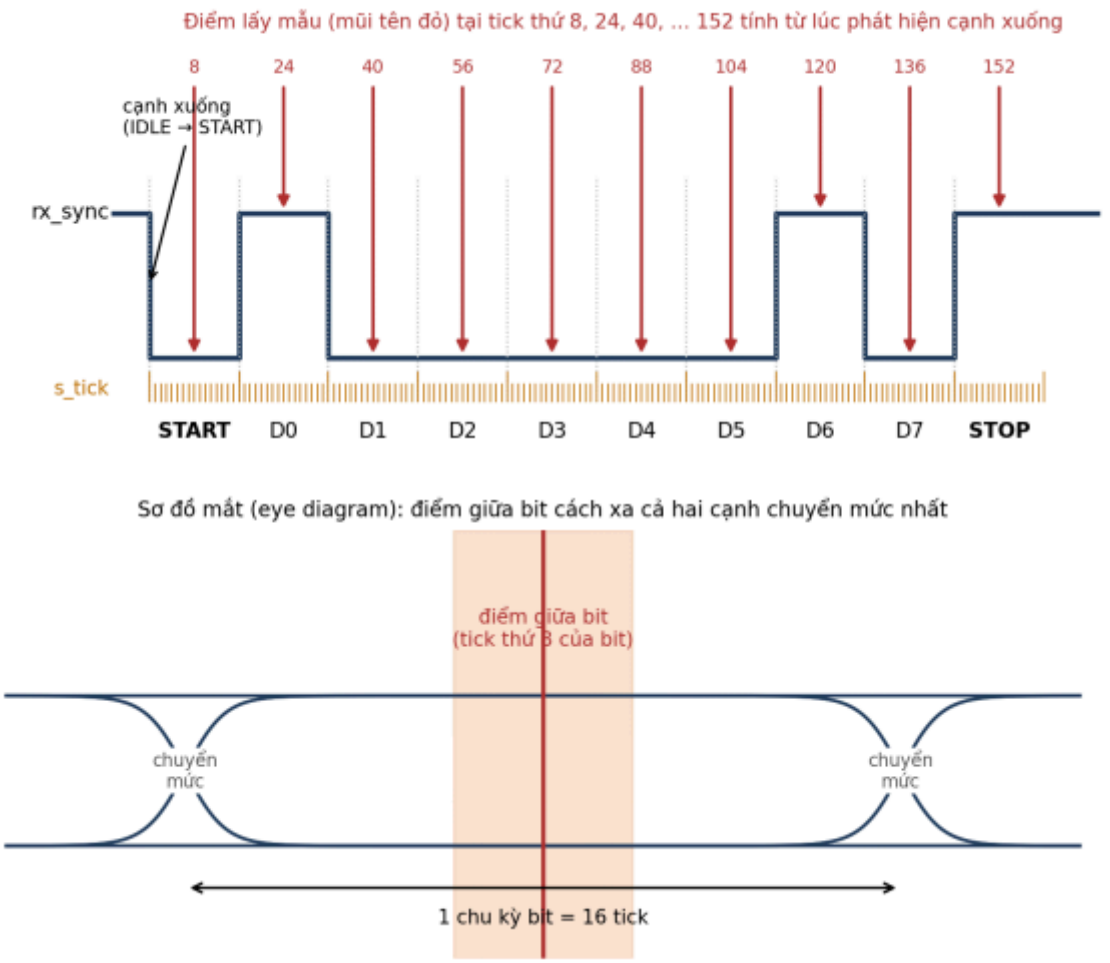
- Xung rx_done_o rộng 1 chu kỳ clk: đầu mỗi chu kỳ rx_done_o <= 1'b0 được gán mặc định, chỉ riêng nhánh STOP hợp lệ ghi đè bằng 1. Vì rx_data_o và rx_done_o cùng được cập nhật tại một cạnh clock, dữ liệu đã ổn định ngay khi xung rx_done tới wr_en của FIFO_RX mà không cần thêm thanh ghi trung gian.

- Ghi bit theo chỉ số: data_reg[bit_count] <= rx_sync đặt bit thứ k vào đúng vị trí k nên với chuẩn LSB first, sau 8 bit data_reg chứa đúng byte gốc mà không cần đảo thứ tự bit.

```
DATA: begin
   if (s_tick_i) begin
      if (tick_count == TICKS_PER_BIT - 1) begin   // đủ 16 tick: giữa bit
         tick_count <= 0;
         data_reg[bit_count] <= rx_sync;          // lấy mẫu
         if (bit_count == DATA_WIDTH - 1) begin
            bit_count <= 0;  state <= STOP;
         end else bit_count <= bit_count + 1'b1;
      end else tick_count <= tick_count + 1'b1;
   end
end
```

3.3.2. Thuật toán định vị điểm giữa bit (Eye-diagram center)

Trong sơ đồ mắt (eye diagram), các đường tín hiệu của nhiều bit được chồng lên nhau trong một cửa sổ bằng 1 chu kỳ bit; vùng "mắt mở" nằm giữa hai cạnh chuyển mức và có biên độ ổn định, nhiễu nhỏ nhất ở chính giữa. Vì vậy thời điểm tốt nhất để lấy mẫu là **giữa chu kỳ bit**, tức tick thứ 8 của mỗi bit (Hình 3.6).



Hình 3.6: Điểm lấy mẫu giữa bit của uart_rx và sơ đồ mắt

Bit	Tick lấy mẫu	Thời điểm (µs)	Giá trị
START	8	4,32	phải bằng 0
D0 (LSB)	24	12,96	data_reg[0]
D1	40	21,60	data_reg[1]
D2	56	30,24	data_reg[2]
D3	72	38,88	data_reg[3]
D4	88	47,52	data_reg[4]
D5	104	56,16	data_reg[5]
D6	120	64,80	data_reg[6]
D7 (MSB)	136	73,44	data_reg[7]
STOP	152	82,08	phải bằng 1 → rxdone_o

Bảng 3.10: Thời điểm lấy mẫu của uart_rx tính từ lúc phát hiện cạnh xuống

Thuật toán định vị:

1) Ở IDLE, ngay khi rx_sync == 0 (cạnh xuống của bit START), FSM sang START và bắt đầu đếm các xung s_tick_i từ 0.

2) Ở START, đếm đến HALF_BIT_TICK - 1 = 7 (tức 8 tick = nửa bit) thì kiểm tra lại: nếu rx_sync vẫn bằng 0 thì đây đúng là bit START (đang ở giữa bit START), chuyển sang DATA; nếu đã bằng 1 thì chỉ là xung nhiễu hẹp, quay về IDLE.

3) Ở DATA, tick_count được xóa về 0 ngay tại giữa bit START, nên mỗi lần đếm đủ 16 tick tiếp theo đều rơi đúng giữa bit kế tiếp (tại tick 24, 40, … tính từ cạnh xuống): lấy mẫu rx_sync và dịch sang bit sau.

4) Ở STOP, thêm 16 tick để tới giữa bit STOP (tick 152) rồi kiểm tra mức 1. Tổng thời gian 8 + 8×16 + 16 = 152 tick = 9,5 chu kỳ bit; bộ thu quay về IDLE ngay giữa bit STOP, còn dư nửa bit trước khi bit START của khung kế tiếp có thể xuất hiện nên có thể thu các khung liên tiếp không có khoảng nghỉ.

Sai số định vị: Vì xung s_tick không đồng bộ pha với thời điểm FSM phát hiện cạnh xuống, tick đầu tiên có thể đến sau 1 đến 27 chu kỳ clk. Do đó điểm lấy mẫu của bit đầu tiên có thể lệch sớm tối đa khoảng 1 tick (≈ 6 % chu kỳ bit); các bit sau cách nhau đúng 16 tick nên giữ nguyên độ lệch này. Cộng với độ trôi do sai số baud (khoảng 4,5 % tại bit STOP, mục 3.2.2), điểm lấy mẫu vẫn nằm trong khoảng 40 %–60 % chu kỳ bit, sâu trong vùng "mắt mở".

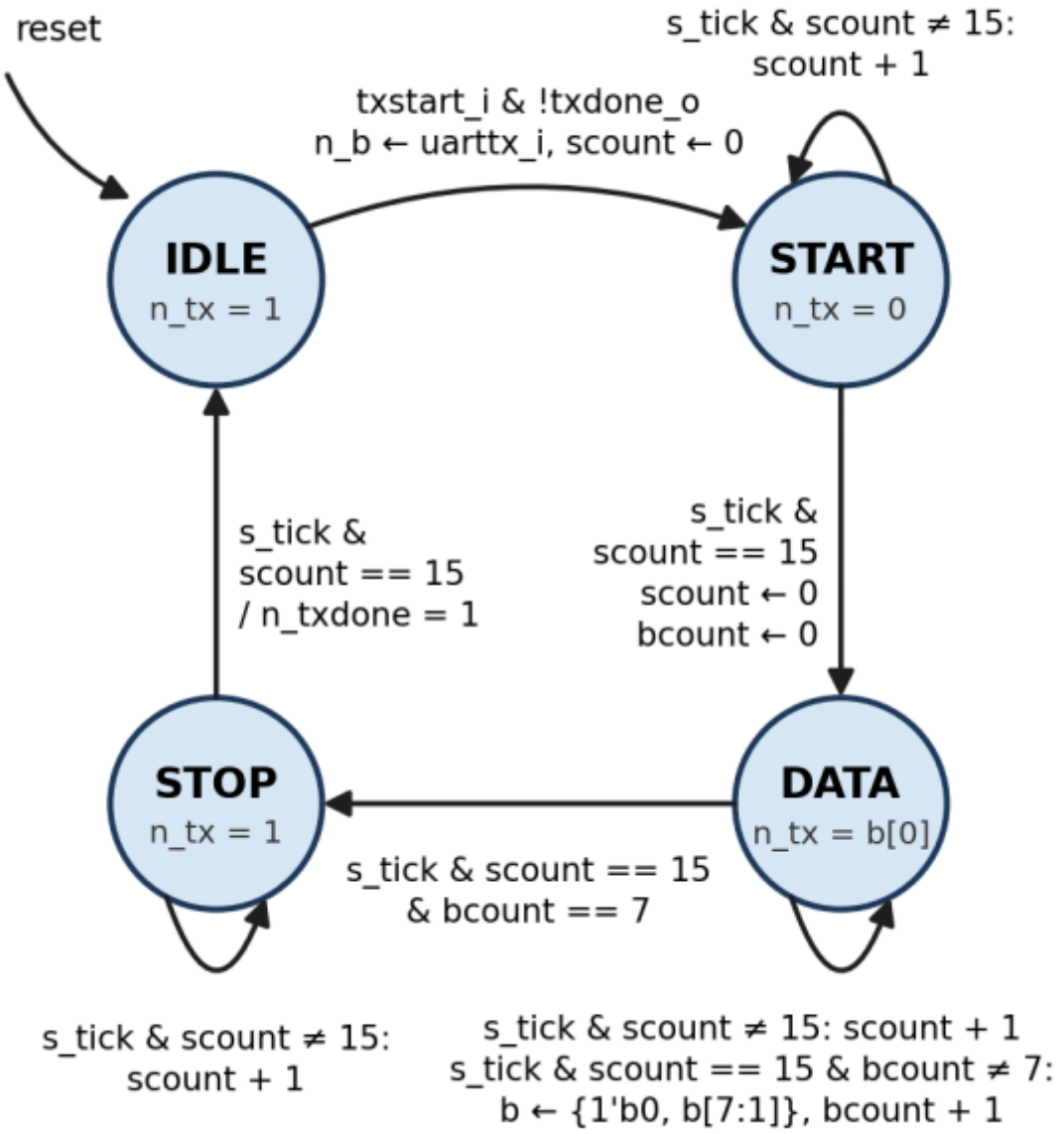
3.4. **Thiết kế chi tiết module phát dữ liệu (UART_TX)**

UART_TX nạp một byte song song từ FIFO_TX và phát thành khung gồm 1 bit START (mức 0), DATA_WIDTH bit dữ liệu (LSB trước), 1 bit STOP (mức 1). Mỗi bit kéo dài đúng 16 xung s_tick (SB_TICK = 16).

Tên tín hiệu	Hướng	Độ rộng (bit)	Chức năng
clk	input	1	Xung nhịp hệ thống.
rst_n	input	1	Reset bất đồng bộ mức thấp: state = IDLE, uarttx_o = 1, txdone_o = 0.
uarttx_i	input	8 bits	Byte cần phát, lấy từ đầu FIFO_TX (tx_data).
s_tick_i	input	1	Xung lấy mẫu từ BAUD_GEN, dùng làm Clock Enable.
txstart_i	input	1	Có dữ liệu chờ phát (~tx_empty).
uarttx_o	output	1	Đường phát nối tiếp (đầu ra thanh ghi).
txdone_o	output	1	Xung rộng 1 chu kỳ clk báo đã phát xong 1 khung.

Bảng 3.11: Cổng giao tiếp của UART_TX

3.4.1. **Sơ đồ máy trạng thái (FSM) và thanh ghi dịch**



Hình 3.7: Sơ đồ FSM của UART_TX

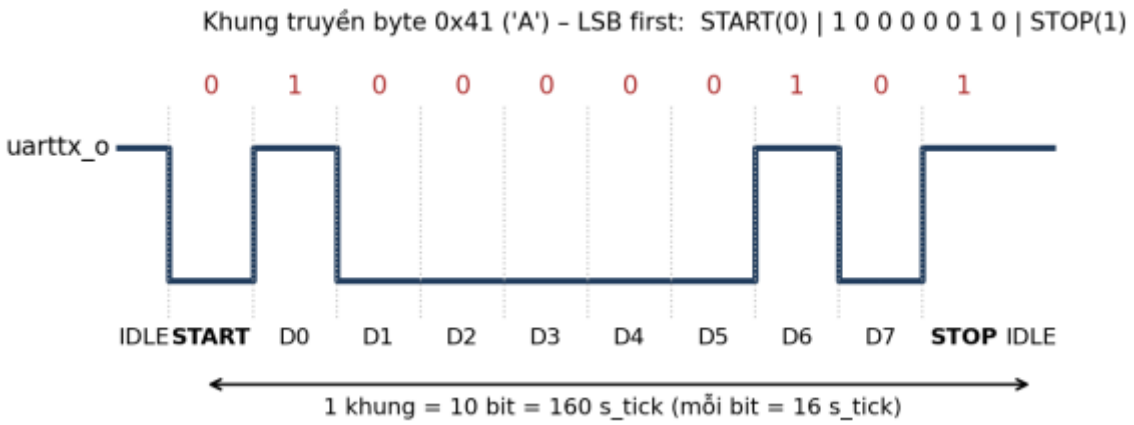
FSM được viết theo đúng mô hình hai khối: khối tổ hợp proc_state (always_comb) tính n_state, n_scount, n_bcount, n_b, n_txdone, n_tx; khối tuần tự

proc_output (always_ff) chép các giá trị này vào state, scount (4 bit, đếm 0–15 tick), bcount (đếm số bit dữ liệu đã phát), b, txdone_o, uarttx_o.

Trạng thái	Mức ra n_tx	Hành động	Điều kiện chuyển
IDLE	1	Giữ đường truyền ở mức nghỉ. Khi txstart_i && !txdone_o, nạp n_b = uarttx_i, xóa scount.	→ START
START	0	Kéo đường truyền xuống 0 trong 16 tick.	s_tick_i & scount == 15 → DATA
DATA	b[0]	Phát bit thấp nhất của b. Cuối mỗi bit (16 tick) dịch phải: n_b = {1'b0, b[DATA_WIDTH-1:1]} và tăng bcount.	s_tick_i & scount == 15 & bcount == 7 → STOP
STOP	1	Giữ mức 1 trong 16 tick; kết thúc thì đặt n_txdone = 1.	s_tick_i & scount == 15 → IDLE

Bảng 3.12: Hoạt động của từng trạng thái trong UART_TX

Thanh ghi dịch: Byte cần phát được chốt vào thanh ghi b ở IDLE. Trong trạng thái DATA, đầu ra tổ hợp n_tx luôn bằng b[0]; sau mỗi bit, b dịch phải một vị trí (chèn 0 vào MSB) nên bit kế tiếp lại xuất hiện ở b[0]. Nhờ đó chỉ cần một dây cố định b[0] thay vì mux 8→1 chọn bit theo bcount, và thứ tự phát tự động là LSB trước. Hình 3.8 minh họa khung truyền byte 0x41.



Hình 3.8: Khung truyền 8N1 của byte 0x41 trên uarttx_o (LSB first)

Thời gian một khung là 16 + 8×16 + 16 = 160 tick = 10 chu kỳ bit = 160 × 27 = 4320 chu kỳ clk (86,4 µs); tốc độ dữ liệu hữu ích khoảng 11,5 KB/s.

Bit	b trước khi dịch	n_tx = b[0]	b sau khi dịch
START	—	0	b = 01000001 (đã nạp)
D0	01000001	1	00100000
D1	00100000	0	00010000
D2	00010000	0	00001000
D3	00001000	0	00000100
D4	00000100	0	00000010
D5	00000010	0	00000001
D6	00000001	1	00000000
D7	00000000	0	00000000
STOP	—	1	—

Bảng 3.13. Diễn biến thanh ghi dịch b khi phát byte 0x41 (01000001b)

Gán ngõ ra n_tx bên ngoài khối if (s_tick_i): Trong mọi trạng thái, n_tx được gán ngoài if (s_tick_i):

```
START: begin
```

```
  n_tx = 1'b0;              // gán NGOÀI if (s_tick_i)
  if (s_tick_i) begin
    if (scount == 4'd15) begin ... end
    else n_scount = scount + 4'b1;
  end
end
```

s_tick_i chỉ xuất hiện mỗi 27 chu kỳ clk. Nếu n_tx được gán **bên trong** if (s_tick_i), mức ra chỉ được cập nhật khi tick tới, còn lúc khác n_tx giữ giá trị cũ. Khi FSM vừa chuyển trạng thái (ví dụ DATA → STOP), đường truyền sẽ giữ mức cũ đến tick kế tiếp, độ trễ 0–26 chu kỳ tùy pha tick, làm bit bị kéo dài hoặc rút ngắn và bit STOP có thể méo hoặc mất. Gán ngoài khối if thì mức ra phản ánh trạng thái ngay chu kỳ kế tiếp, ổn định suốt 16 tick của bit và độ rộng bit không phụ thuộc pha của s_tick. Ngoài ra uarttx_o <= n_tx là đầu ra thanh ghi nên không có glitch tổ hợp.

3.4.2. Xử lý khoảng nghỉ an toàn tại trạng thái IDLE

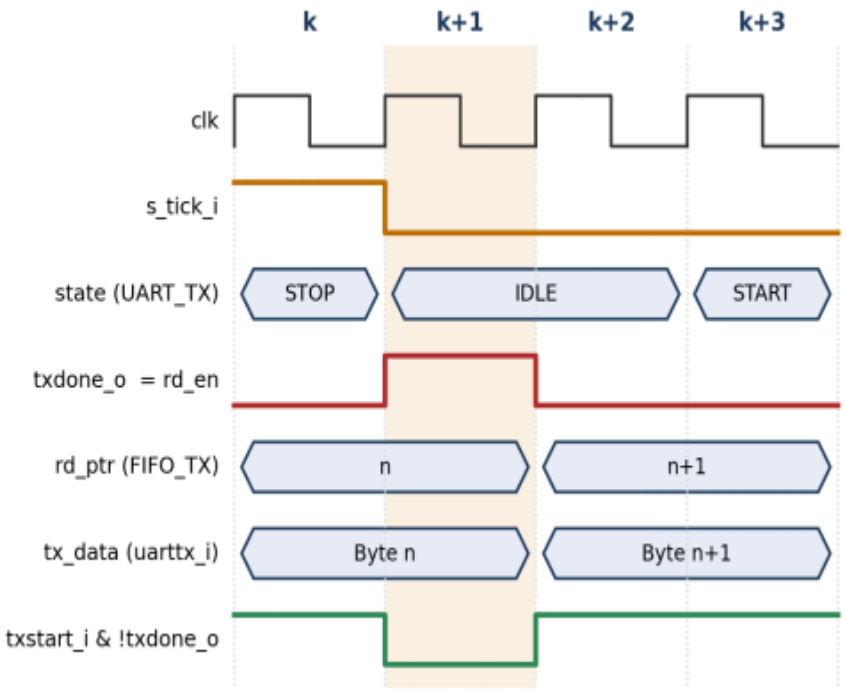
```
IDLE: begin
  n_tx = 1'b1;
  if (txstart_i && !txdone_o) begin   // chốt chặn: !txdone_o
    n_state  = START;
    n_b      = uarttx_i;
    n_scount = 4'd0;
  end
end
```

txdone_o là tín hiệu có thanh ghi, nên nó lên 1 ở chu kỳ ngay sau khi FSM rời STOP về IDLE; cũng ở chu kỳ đó FIFO_TX nhận rd_en = tx_done nhưng con trỏ đọc chỉ thực sự tăng ở cạnh clock kế tiếp. Nếu IDLE chỉ kiểm tra txstart_i, thì trong chu kỳ này tx_empty chưa cập nhật và tx_data vẫn là byte vừa phát, FSM sẽ hiểu nhầm là có dữ liệu mới. Diễn biến theo từng chu kỳ clock:

Chu kỳ	state	txdone_o	FIFO_TX	Kết quả
k	STOP (scount = 15, tick)	0	Chưa đọc	n_txdone = 1, n_state = IDLE
k+1	IDLE	1	rd_en = 1: cạnh clock cuối chu kỳ tăng rd_ptr, giảm	Chốt chặn !txdone_o = 0 ⇒ **ở lại IDLE**

			count. tx_start vẫn = 1 (chưa cập nhật)	
k+2	IDLE	0	rd_ptr, count đã cập nhật; tx_start, tx_data phản ánh byte kế tiếp (hoặc rỗng)	Nếu còn dữ liệu: nạp byte mới, sang START

Bảng 3.14: Diễn biến quanh thời điểm kết thúc một khung truyền



Hình 3.9: Giản đồ thời gian chốt chặn !txdone_o tại IDLE

Với chốt chặn, bộ phát nghỉ đúng 1 chu kỳ clock giữa hai khung để FIFO_TX kịp cập nhật con trỏ đọc và cờ tx_empty; cái giá chỉ là 20 ns, không đáng kể so với 86,4 µs của một khung.

3.5. **Thiết kế chi tiết module bộ đệm (FIFO FWFT)**

FIFO là bộ đệm đồng bộ một clock, được dùng hai lần trong UART_TOP (u_fifo_rx, u_fifo_tx). Độ sâu 2^ADDR_WIDTH = 16 ô, mỗi ô rộng DATA_WIDTH = 8 bit.

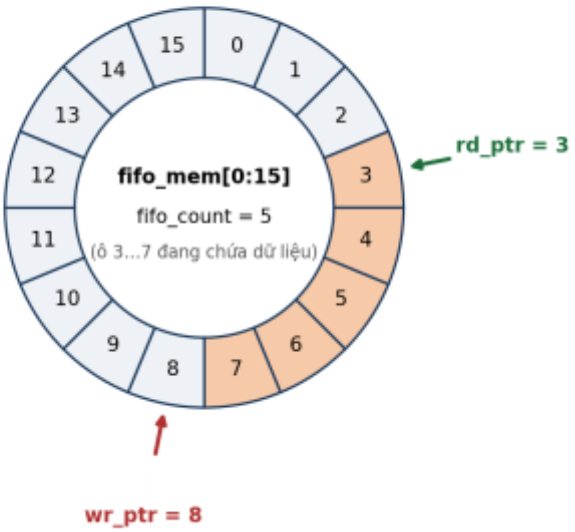
Tên tín hiệu	Hướng	Độ rộng (bit)	Chức năng
clk	input	1	Xung nhịp, dùng chung cho đọc và ghi.
rst_n	input	1	Reset bất đồng bộ mức thấp: xóa wr_ptr, rd_ptr, fifo_count.
wr_en	input	1	Cho phép ghi; chỉ có tác dụng khi FIFO chưa đầy.
wr_data_i	input	DATA_WIDTH (8)	Dữ liệu cần ghi.
rd_en	input	1	Lệnh lấy phần tử đầu hàng đợi ra (tăng con trỏ đọc); chỉ có tác dụng khi FIFO không rỗng.
rd_data_o	output	DATA_WIDTH (8)	Phần tử đầu hàng đợi, luôn có sẵn (FWFT).
full_o	output	1	FIFO đầy.
empty_o	output	1	FIFO rỗng.

Bảng 3.15: Cổng giao tiếp của FIFO

3.5.1. Quản lý bộ đệm vòng (Circular Buffer) và cờ báo trạng thái

- **Mảng nhớ** fifo_mem[0:15] gồm 16 ô, mỗi ô 8 bit.
- **Hai con trỏ 4 bit** wr_ptr (ô sẽ ghi tiếp theo) và rd_ptr (ô ở đầu hàng đợi). Vì rộng đúng ADDR_WIDTH = 4 bit nên khi tăng vượt ô 15 chúng tự tràn về 0, tạo thành bộ đệm vòng mà không cần phép so sánh hay lệnh reset con trỏ.

- **Thanh ghi đếm 5 bit** fifo_count ([ADDR_WIDTH:0]) lưu số phần tử đang có, từ 0 đến 16; cần 5 bit vì phải biểu diễn được giá trị 16 khi đầy.



Hình 3.10: Bộ đệm vòng 16 ô với wr_ptr, rd_ptr và fifo_count

Hoạt động cập nhật gồm khối tổ hợp tính next_wr_ptr, next_rd_ptr, next_fifo_count và một khối always_ff có reset để chép vào thanh ghi. Ghi và đọc được xét có điều kiện wr_en && !full_o và rd_en && !empty_o, nên ghi vào FIFO đầy hoặc đọc từ FIFO rỗng đều bị bỏ qua an toàn:

Ghi hợp lệ / Đọc hợp lệ	wr_ptr	rd_ptr	fifo_count
0 / 0	giữ	giữ	giữ
1 / 0 (chỉ ghi)	+1	giữ	+1
0 / 1 (chỉ đọc)	giữ	+1	−1
1 / 1 (đọc và ghi đồng thời)	+1	+1	giữ nguyên

Bảng 3.16: Quy tắc cập nhật con trỏ và bộ đếm

Bước	Thao tác	wr_ptr	rd_ptr	count	rd_data_o / cờ
1	Sau reset	0	0	0	empty_o = 1
2	Ghi byte A	1	0	1	A
3	Ghi byte B	2	0	2	A
4	Đọc (pop A) và ghi C đồng thời	3	1	2	B
5	Đọc (pop B)	3	2	1	C
6	Đọc (pop C)	3	3	0	empty_o = 1

Bảng 3.17: Ví dụ diễn biến FIFO (ADDR_WIDTH = 4)

Cờ báo trạng thái:

```
assign empty_o = (fifo_count == 0);
assign full_o  = (fifo_count == (1 << ADDR_WIDTH));   // = 16
```

Cờ được suy ra từ bộ đếm số phần tử nên không cần so sánh hai con trỏ, do đó tận dụng tối đa cả 16 ô nhớ. Với phương pháp so sánh con trỏ thuần túy, trạng thái đầy và rỗng đều có wr_ptr == rd_ptr nên phải bỏ phí một ô (chỉ chứa 15 phần tử); phương án thêm một bit vào mỗi con trỏ thì phải so sánh nhiều bit. Với bộ đếm, empty_o chỉ là so sánh với 0, còn vì fifo_count không vượt quá 16 nên full_o thực chất là bit cao nhất fifo_count[4]; cái giá phải trả là một bộ đếm lên/xuống 5 bit nhỏ.

Tối ưu tổng hợp cho FPGA. Mảng nhớ được tách thành một khối always_ff @(posedge clk) độc lập, không chứa `rst_n`:

```
always_ff @(posedge clk) begin          // không có reset
    if (wr_en && !full_o)
        fifo_mem[wr_ptr] <= wr_data_i;
end
```

- Các ô nhớ RAM trong FPGA không có chân reset từng ô. Nếu đặt fifo_mem chung trong khối có reset (for (...) fifo_mem[i] <= 0), công cụ tổng hợp không suy luận được RAM mà phải xây mảng từ Flip-Flop rời, mỗi FF kèm logic reset và enable.

- Khối ghi đồng bộ cùng với đọc bất đồng bộ assign rd_data_o = fifo_mem[rd_ptr] khớp mô hình RAM ghi đồng bộ – đọc bất đồng bộ của LUTRAM (Distributed RAM), nên được ánh xạ thẳng vào LUT dùng làm bộ nhớ.

- Không cần reset nội dung RAM vì empty_o (suy từ fifo_count, có reset) đã ngăn việc đọc ô chưa ghi.

Tiêu chí	Mảng nhớ có reset (chung khối)	Mảng nhớ tách riêng, không reset
Cách ánh xạ	Flip-Flop rời + mux đọc 16:1	LUTRAM / Distributed RAM
Flip-Flop cho mảng nhớ	16 × 8 = 128 FF mỗi FIFO (≈ 256 FF cho 2 FIFO)	0 FF
Tài nguyên logic	Logic reset/enable cho từng FF, mux đọc	Vài chục LUT dùng làm RAM
FF còn lại mỗi FIFO	13 FF (con trỏ 4+4, bộ đếm 5)	13 FF (con trỏ 4+4, bộ đếm 5)

Bảng 3.18: So sánh hai cách mô tả mảng nhớ (ước lượng cho 2 FIFO 16 × 8)

Kết quả là tiết kiệm hàng trăm Flip-Flop rời (cỡ 256 FF cho hai FIFO). Kỹ thuật này gắn với độ sâu nhỏ; nếu tăng ADDR_WIDTH rất lớn thì nên chuyển sang Block RAM với đọc đồng bộ. Khi mô phỏng, trước lần ghi đầu tiên rd_data_o có thể hiển thị X; điều này hợp lệ vì khi đó FIFO rỗng và dữ liệu không được sử dụng.

3.5.2: Giải quyết xung đột đọc/ghi đồng thời (Read-During-Write Hazard)

Kiến trúc FWFT (First-Word Fall-Through). Phần tử cũ nhất luôn xuất hiện sẵn trên rd_data_o ngay khi FIFO không rỗng; chân rd_en mang ý nghĩa "đã lấy xong phần tử đầu, chuyển sang phần tử kế" (pop). Thiết kế mô tả bằng mạch tổ hợp thuần assign rd_data_o = fifo_mem[rd_ptr] thay vì Flip-Flop đọc đồng bộ, vì:

- **Không có độ trễ đọc**: đọc đồng bộ cho dữ liệu sau 1 chu kỳ kể từ khi rd_ptr đổi; muốn có hành vi FWFT phải thêm logic đọc trước (prefetch), thanh ghi đầu ra và cờ hợp lệ.

- **Tiết kiệm tài nguyên**: không cần thanh ghi đầu ra 8 bit; rd_ptr đóng vai trò thanh ghi địa chỉ.

- **Phù hợp UART_TOP**: en_transfer điều khiển đồng thời rd_en của FIFO_RX và wr_en của FIFO_TX nên dữ liệu phải sẵn sàng trên transfer_data ngay trong chu kỳ đó.

Phân tích hazard khi count == 1. Xét trường hợp nguy hiểm nhất: FIFO có đúng 1 phần tử và trong cùng một chu kỳ vừa đọc (rút phần tử đó) vừa ghi (đưa phần tử mới vào). Giả sử rd_ptr = 3, wr_ptr = 4, dữ liệu mới là D:

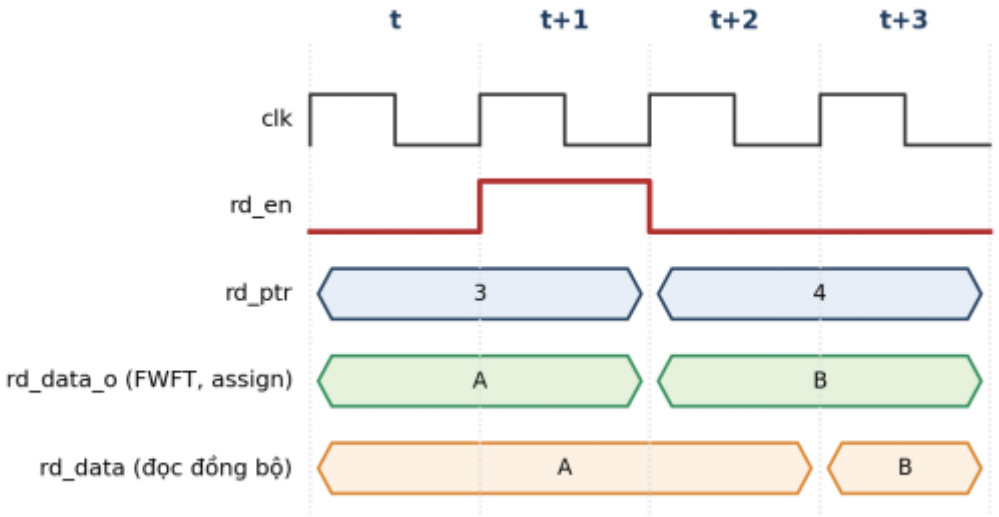
Thời điểm	Ghi	Đọc
Chu kỳ t (trước cạnh clk)	fifo_mem[4] <= D (ô 4 khác ô 3)	rd_data_o = fifo_mem[3] (phần tử cũ), rd_ptr ← 4
Chu kỳ t+1	wr_ptr = 5, count = 1	rd_ptr = 4 ⇒ rd_data_o = fifo_mem[4] = D (phần tử mới)

Bảng 3.18: So sánh hai cách mô tả mảng nhớ

- Ô đang đọc (rd_ptr) và ô đang ghi (wr_ptr) luôn khác nhau khi FIFO không rỗng và không đầy, nên không có truy cập đồng thời vào cùng một ô. Hai trường hợp wr_ptr == rd_ptr là rỗng (không đọc được vì !empty_o) và đầy (không ghi được vì !full_o).

- Việc ghi fifo_mem[wr_ptr] và việc tăng rd_ptr lên đúng ô đó xảy ra **trong cùng một cạnh clock**. Sau cạnh clock, rd_ptr đã trỏ tới ô vừa ghi và fifo_mem[rd_ptr] đã chứa D, nên rd_data_o đúng ngay mà không cần mạch chuyển tiếp dữ liệu (bypass).

- Với FIFO đọc đồng bộ, phần tử kế tiếp phải được lấy sẵn trong chu kỳ t nhưng khi đó D chưa nằm trong bộ nhớ, nên phải có logic so sánh địa chỉ và chuyển tiếp dữ liệu để khỏi đọc phải dữ liệu cũ; đó chính là hazard mà đọc tổ hợp của FWFT loại bỏ.



Hình 3.11. So sánh đọc tổ hợp FWFT và đọc đồng bộ khi rút một phần tử

3.6. Module UART_TOP và cơ chế ghép nối vòng lặp tự động (Auto-Loopback)

UART_TOP ghép năm thực thể theo Hình 3.1 và thêm hai phép gán tổ hợp tạo nên cơ chế loopback tự động mà không cần bộ điều khiển (FSM) riêng:

```
assign tx_start    = ~tx_empty;                // FIFO_TX có dữ liệu → phát
assign en_transfer = (~rx_empty) & (~tx_full);  // chuyển FIFO_RX → FIFO_TX
```

Tín hiệu	Độ rộng	Nguồn → Đích	Mô tả
s_tick	1	u_baud_gen.s_tick_o → u_uart_rx.s_tick_i, u_uart_tx.s_tick_i	Xung lấy mẫu rộng 1 chu kỳ clk, tần số 16 × baud.
rx_done	1	u_uart_rx.rx_done_o → u_fifo_rx.wr_en	Xung báo đã nhận xong 1 byte hợp lệ, kích ghi vào FIFO_RX.
rx_data	[7:0]	u_uart_rx.rx_data_o → u_fifo_rx.wr_data_i	Byte vừa nhận được.
rx_empty	1	u_fifo_rx.empty_o → cổng AND	Cờ rỗng của FIFO_RX.
transfer_data	[7:0]	u_fifo_rx.rd_data_o → u_fifo_tx.wr_data_i	Dữ liệu đầu hàng đợi FIFO_RX (FWFT), đưa thẳng sang FIFO_TX.

en_transfer	1	cổng AND → u_fifo_rx.rd_en, u_fifo_tx.wr_en	~rx_empty & ~tx_full: cho phép chuyển 1 byte.
tx_full	1	u_fifo_tx.full_o → cổng AND	Cờ đầy của FIFO_TX, chặn việc chuyển khi không còn chỗ.
tx_empty	1	u_fifo_tx.empty_o → cổng NOT	Cờ rỗng của FIFO_TX.
tx_start	1	cổng NOT → u_uart_tx.txstart_i	~tx_empty: có dữ liệu chờ phát.
tx_data	[7:0]	u_fifo_tx.rd_data_o → u_uart_tx.uarttx_i	Byte đầu hàng đợi FIFO_TX cần phát.
tx_done	1	u_uart_tx.txdone_o → u_fifo_tx.rd_en	Xung báo phát xong 1 khung, rút byte đã phát khỏi FIFO_TX.

Bảng 3.20: Danh sách các dây tín hiệu nội bộ của UART_TOP

Quy ước tên tín hiệu theo sơ đồ: các tên có hậu tố _i và _o là cổng vào/ra của module; các tên không có hậu tố là dây nối nội bộ tại UART_TOP. Cụ thể, rx_done_o và rx_data_o của UART_RX được nối thành rx_done và rx_data; txstart_i và uarttx_i của UART_TX lần lượt nhận tx_start và tx_data; txdone_o được nối thành tx_done. Tín hiệu s_tick_o của BAUD_GEN được nối thành s_tick và cấp cho s_tick_i của UART_RX, UART_TX.

Cổng full_o của u_fifo_rx được để hở; tín hiệu full_o khai báo trong UART_TOP chưa được sử dụng.

Nguyên lý hoạt động:

1) u_uart_rx thu xong một khung hợp lệ thì phát xung rx_done kèm rx_data, ghi vào u_fifo_rx.

2) Khi u_fifo_rx không rỗng và u_fifo_tx chưa đầy (en_transfer = 1), trong cùng một chu kỳ clock byte ở đầu u_fifo_rx (transfer_data) được ghi vào u_fifo_tx và đồng thời được rút khỏi u_fifo_rx.

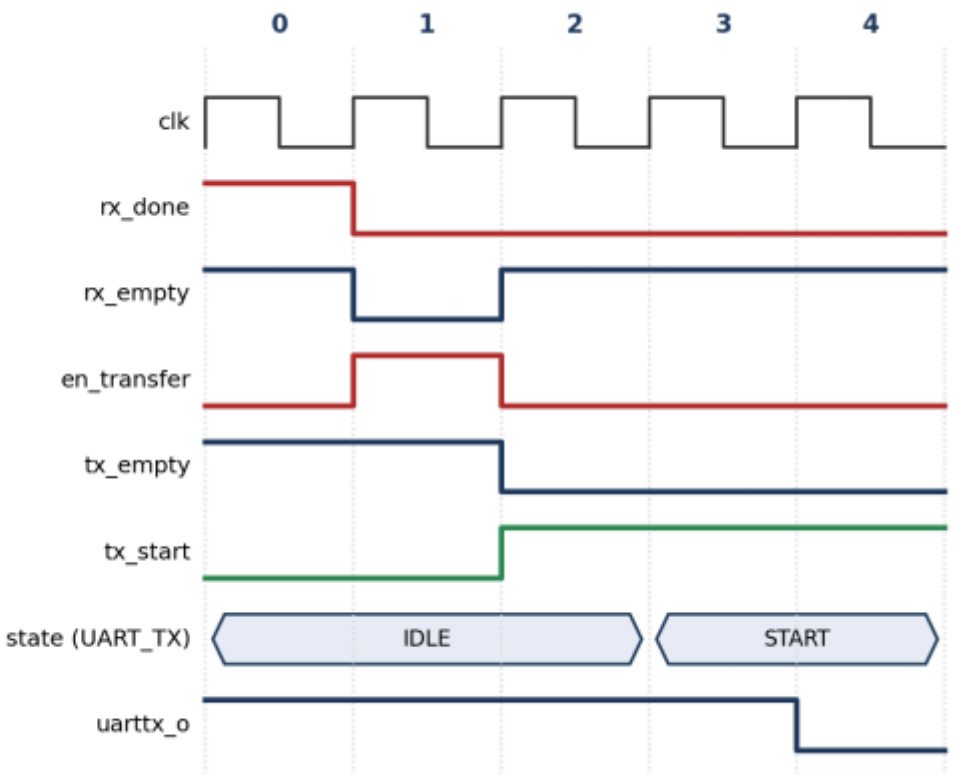
3) Khi u_fifo_tx có dữ liệu (tx_start = 1), u_uart_tx nạp byte tx_data và phát khung 8N1 ra uarttx_o. Kết thúc khung, tx_done rút byte vừa phát khỏi u_fifo_tx, chuẩn bị cho byte kế tiếp.

Dữ liệu đi qua hai tầng FIFO một cách gần như tức thời nhờ đầu ra tổ hợp của FWFT, mỗi tầng chỉ tốn đúng 1 chu kỳ clock:

Chu kỳ	Sự kiện
0	rx_done = 1; cạnh clock cuối chu kỳ ghi byte vào u_fifo_rx.
1	rx_empty = 0 ⇒ en_transfer = 1; transfer_data đã là byte vừa nhận; cạnh clock cuối chu kỳ ghi byte vào u_fifo_tx và rút khỏi u_fifo_rx.
2	tx_empty = 0 ⇒ tx_start = 1; tx_data đã là byte đó; UART_TX ở IDLE nạp byte và sang START.
3	UART_TX ở trạng thái START (n_tx = 0); uarttx_o là đầu ra thanh ghi nên xuống 0 ở chu kỳ kế tiếp (bit START bắt đầu).

Bảng 3.21: Độ trễ truyền dữ liệu qua hai tầng FIFO

Từ lúc nhận xong byte đến lúc bắt đầu phát lại chỉ mất khoảng 3–4 chu kỳ clock (≈ 60–80 ns). Nếu dùng FIFO đọc đồng bộ, mỗi tầng sẽ tốn thêm ít nhất 1 chu kỳ cùng các thanh ghi trung gian. Nhờ cờ empty/full và các xung done làm tín hiệu bắt tay, các khối không cần tín hiệu yêu cầu/xác nhận riêng; FIFO tách tốc độ giữa phía thu và phía phát, còn điều kiện ~tx_full bảo đảm không ghi tràn FIFO_TX.



Hình 3.12: Giản đồ thời gian chuyển dữ liệu

Mốc	Thời gian từ cạnh xuống START của khung nhận	Ghi chú
Bộ thu lấy mẫu bit STOP, phát rx_done	≈ 82,08 µs	tick 152 (9,5 bit)
Byte tới u_uart_tx, bắt đầu START phát lại	≈ 82,14 µs	thêm 3–4 chu kỳ clk
Kết thúc khung phát lại	≈ 168,5 µs	bit START + 10 bit

Bảng 3.22: Bảng cân đối thời gian của một lượt echo

Bộ phát và bộ thu dùng chung một BAUD_GEN, nên tốc độ nhận và phát byte bằng nhau (mỗi byte 10 bit). Mỗi byte chỉ nằm trong FIFO khoảng vài chu kỳ clock trước khi được chuyển, và sau đó chờ UART_TX rảnh; với luồng liên tục các khung 8N1, số byte tồn đọng trong hai FIFO không vượt quá 2, nhỏ hơn nhiều so với độ sâu 16.

Khối	Chức năng	Kỹ thuật RTL nổi bật
UART_TOP	Ghép 5 thực thể, điều khiển loopback RX → FIFO → FIFO → TX	tx_start = ~tx_empty; en_transfer = ~rx_empty & ~tx_full
BAUD_GEN	Tạo s_tick = 16 × baud	Bộ đếm tuần hoàn, xung 1 chu kỳ clk; sai số 0,469 % < 2 %
uart_rx	Thu khung 8N1	Đồng bộ 2 FF, lấy mẫu giữa bit, kiểm tra bit STOP, data_reg[bit_count] <= rx_sync
UART_TX	Phát khung 8N1	FSM 2 khối, thanh ghi dịch b; n_tx gán ngoài if (s_tick_i); chốt chặn !txdone_o
FIFO	Bộ đệm 16 × 8 bit	Bộ nhớ vòng, cờ từ fifo_count, FWFT, mảng nhớ không reset (LUTRAM)

Bảng 3.23: Tóm tắt các khối và kỹ thuật RTL chính

CHƯƠNG 4. **KẾT QUẢ KIỂM THỬ VÀ MÔ PHỎNG HỆ THỐNG**

4.1. **Môi trường kiểm thử Self-Checking Testbench**

Môi trường kiểm thử gồm testbench UART_TOP_TB.sv, thiết kế cần kiểm tra (DUT) UART_TOP, các vector tham chiếu do Python tạo và khối monitor/scoreboard. Testbench tạo clock 50 MHz (chu kỳ 20 ns), reset DUT rồi áp dụng các khung UART 8-N-1 tại ngõ vào uartrx_i.

Với CLK_FREQ = 50 MHz, BAUD_RATE = 115.200 bps và oversampling 16x, bộ chia nguyên của bộ tạo baud là:

$$\text{Baud Rate} = \frac{\text{System Clock Frequency}}{\text{Prescaler} \times 16}$$

Do đó chu kỳ bit trong stimulus là 20 ns × 27 × 16 = 8.640 ns. Baud thực tế xấp xỉ 115.740,74 bps, sai lệch khoảng +0,47% so với baud mục tiêu.

4.1.1. Khởi tạo Stimulus Generator và tích hợp Python Golden Model

Task send_byte tạo một khung gồm Start bit mức thấp, tám bit dữ liệu truyền LSB-first và Stop bit mức cao. Kịch bản kiểm thử gồm hai lần truyền đơn (0xA5, 0x5A) và một burst ba byte liên tiếp (0x31, 0x32, 0x33).

Script GOLDEN_MODEL.py tạo dữ liệu tham chiếu và ghi năm byte kỳ vọng vào golden_vectors.hex. Testbench nạp file này bằng $readmemh vào mảng expected_mem; các vector lần lượt là A5, 5A, 31, 32, 33. Python chỉ tạo mô hình/vector tham chiếu; việc xác minh RTL được thực hiện bằng cách chạy SystemVerilog testbench trên DUT trong QuestaSim.

```
VSIM 2> run -all
# ========================================
#  START UART_TOP ECHO LOOPBACK TEST
#  Loaded 5 bytes from Golden Model
#  BIT_PERIOD = 8640 ns
# ========================================
# [600000 ns] Test 1: Send 0xA5
# [87020000 ns] SENT: 0xa5
# [164430000 ns] PASS: Byte #1 | Expected: 0xa5 | Received: 0xa5
# [216620000 ns] Test 2: Send 0x5A
# [303020000 ns] SENT: 0x5a
# [380430000 ns] PASS: Byte #2 | Expected: 0x5a | Received: 0x5a
# [432620000 ns] Test 3: Burst 0x31, 0x32, 0x33
# [519020000 ns] SENT: 0x31
# [596430000 ns] PASS: Byte #3 | Expected: 0x31 | Received: 0x31
# [605420000 ns] SENT: 0x32
# [682830000 ns] PASS: Byte #4 | Expected: 0x32 | Received: 0x32
# [691820000 ns] SENT: 0x33
# [769230000 ns] PASS: Byte #5 | Expected: 0x33 | Received: 0x33
# ========================================
#  TEST SUMMARY
#  PASS COUNT : 5 / 5
#  ERROR COUNT: 0
#  >>> UART_TOP TEST PASSED <<<
# ========================================
# ** Note: $finish    : C:/Users/Legion/Desktop/KY_4/FAP201/TONG_HOP_CODE_UART/SRC/UART_TOP_TB.sv(136)
#    Time: 1037420 ns  Iteration: 0  Instance: /tb_UART_TOP
# 1
# Break in Module tb_UART_TOP at C:/Users/Legion/Desktop/KY_4/FAP201/TONG_HOP_CODE_UART/SRC/UART_TOP_TB.sv line 136
```

Hình 4.1: Transcript tổng kết mô phỏng: PASS 5/5, ERROR 0.

4.1.2. Mạch giám sát tự động (Monitor) và thuật toán chấm điểm (Scoreboard)

Monitor chờ cạnh xuống trên uarttx_o, đợi nửa chu kỳ bit để xác nhận Start bit, rồi lấy mẫu tám bit dữ liệu tại giữa mỗi bit theo thứ tự LSB-first. Byte thu được được so sánh với vector kỳ vọng tương ứng.

Scoreboard tăng pass_count khi byte khớp; tăng error_count khi byte sai hoặc xuất hiện byte TX ngoài số lượng vector kỳ vọng. Ở cuối mô phỏng, test chỉ báo thành

công nếu không có lỗi, số byte PASS bằng số vector kỳ vọng và số vector lớn hơn 0. Vì vậy, thiếu byte cũng khiến điều kiện tổng kết thất bại. Monitor hiện chưa kiểm tra độc lập giá trị Stop bit hoặc các cờ lỗi nội bộ của FIFO.

4.2. **Kết quả mô phỏng trên phần mềm QuestaSim**

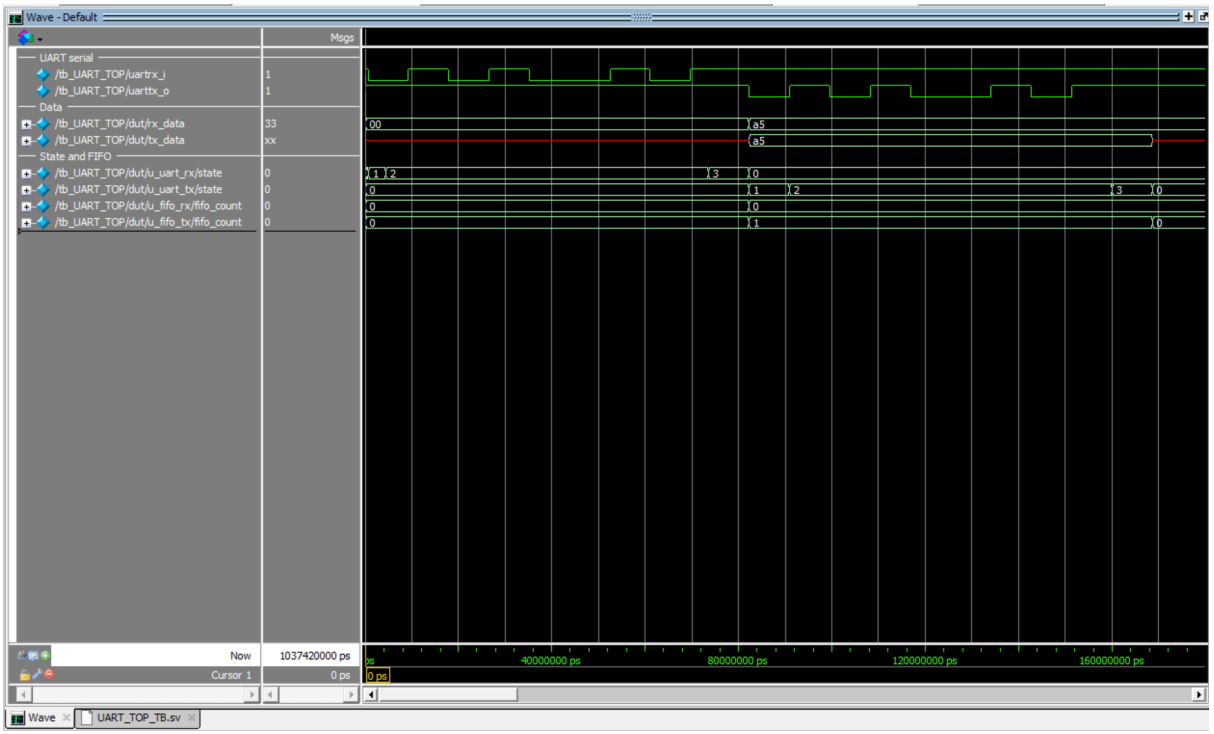
RTL và testbench đã được biên dịch, mô phỏng bằng Questa Altera Starter FPGA Edition 2025.3. Kết quả biên dịch là 0 lỗi, 0 cảnh báo; thời gian mô phỏng đến khi kết thúc là 1.037.420 ns. Cả năm byte truyền ra đều khớp vector tham chiếu:

Lần kiểm thử	Byte gửi	Byte nhận tại uarttx_o	Kết quả
1	0xA5	0xA5	PASS
2	0x5A	0x5A	PASS
3	0x31	0x31	PASS
4	0x32	0x32	PASS
5	0x33	0x33	PASS

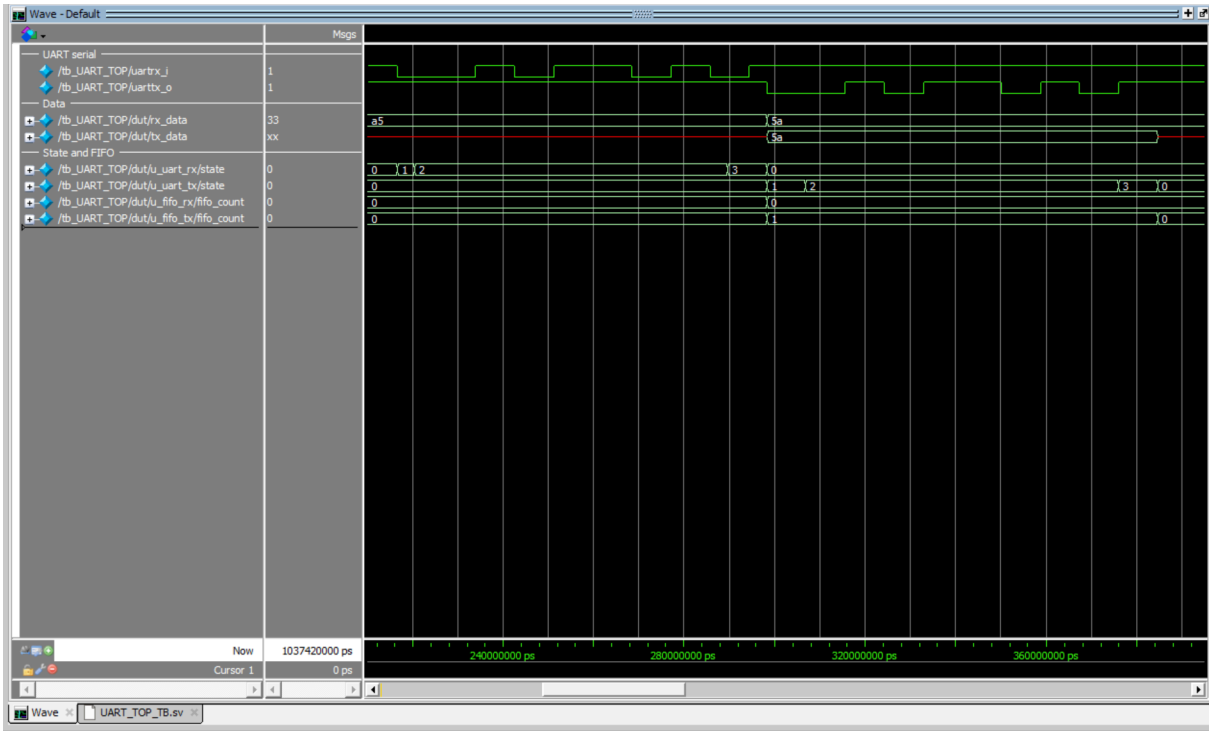
Bảng 4.1: Tổng kết kết quả kiểm thử các trường hợp truyền byte.

4.2.1. Phân tích dạng sóng kịch bản truyền đơn lẻ (Single-Byte Echo)

Ở mỗi trường hợp truyền đơn, uartrx_i mang khung 8-N-1 của byte được gửi; sau khi RX hoàn tất nhận, dữ liệu được chuyển qua các FIFO và UART_TX phát lại byte trên uarttx_o. Đối chiếu dạng sóng với bảng kết quả cho thấy byte echo giữ nguyên giá trị: 0xA5 → 0xA5 và 0x5A → 0x5A.



Hình 4.2: Dạng sóng kiểm thử truyền đơn byte 0xA5.



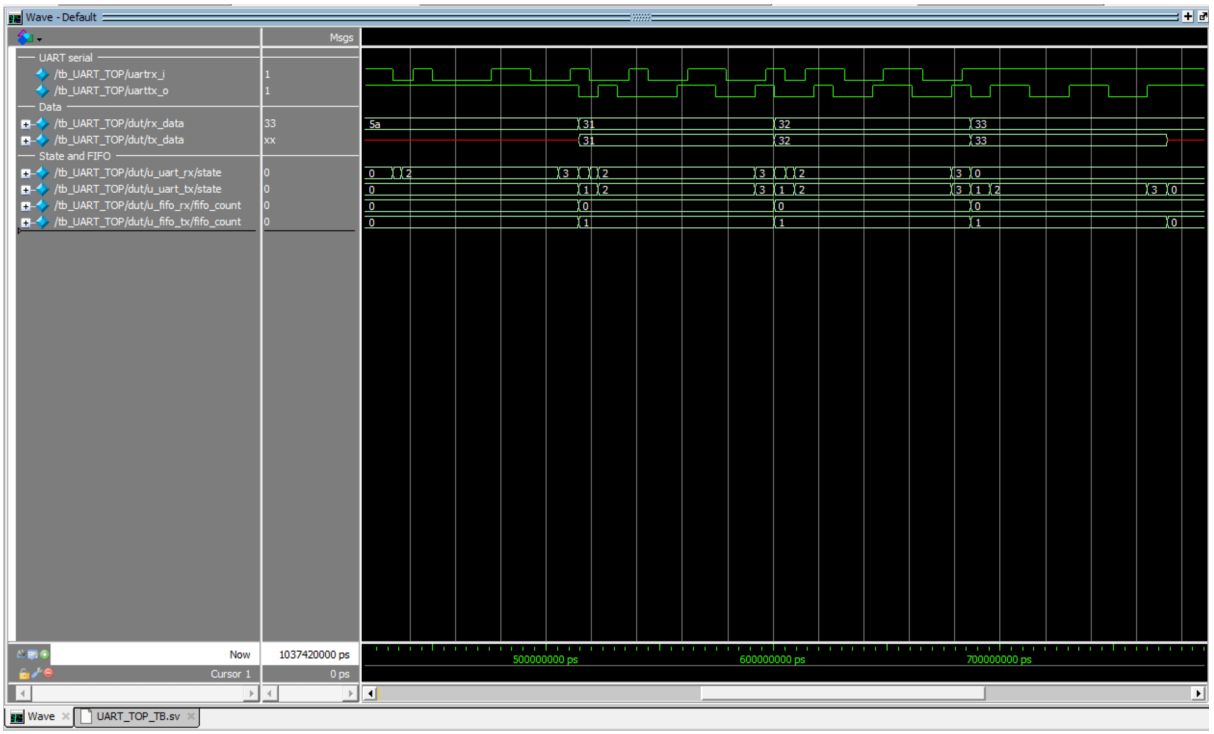
Hình 4.3: Dạng sóng kiểm thử truyền đơn byte 0x5A.

Khi chụp waveform, nên hiển thị tối thiểu uartrx_i, uarttx_o, trạng thái FSM RX/TX và bộ đếm FIFO. Có thể bỏ transfer_data nếu tín hiệu này hiển thị X trong lúc

FIFO rỗng; dữ liệu đó chỉ có ý nghĩa khi FIFO RX có phần tử.

4.2.2. Phân tích dạng sóng kịch bản truyền dồn dập (Burst Stress Test)

Trong burst, testbench gửi liên tiếp 0x31, 0x32, 0x33, mỗi khung bắt đầu cách nhau một chu kỳ khung 86.400 ns. Khi RX nhận byte thứ hai 0x32, TX phát echo byte thứ nhất 0x31; tiếp đó, khi RX nhận byte thứ ba 0x33, TX phát echo byte thứ hai 0x32. Hai tín hiệu nối tiếp uartrx_i và uarttx_o, cùng trạng thái FSM RX/TX và bộ đếm FIFO, thể hiện sự chồng lấp giữa hoạt động nhận và phát. Transcript cũng xác nhận ba byte echo trả về đúng thứ tự và giá trị.



Hình 4.4: Dạng sóng burst 0x31, 0x32, 0x33 thể hiện sự chồng lấp RX/TX.

Đây là burst ngắn ở cùng tốc độ baud với đường truyền TX, dùng để kiểm tra việc nhận và echo liên tiếp không mất byte. Kịch bản này không làm FIFO đầy, không đo thông lượng cực đại và không kiểm tra trường hợp tốc độ dữ liệu vào vượt tốc độ phát. Do đó, kết quả hỗ trợ nhận định về hoạt động liên tục có đệm FIFO nhưng chưa đủ để kết luận hệ thống đã vượt qua stress test dung lượng FIFO.

4.3. **Đánh giá độ tin cậy và khả năng chống nghẽn của hệ thống**

Với cấu hình mặc định và các trường hợp đã chạy, hệ thống trả echo chính xác 5/5 byte, không phát sinh lỗi scoreboard; cả hai kịch bản truyền đơn và chuỗi burst ngắn đều hoạt động đúng. Mô phỏng burst cho thấy bộ thu có thể tiếp nhận khung kế tiếp trong khi bộ phát đang gửi echo của byte trước, còn FIFO làm nhiệm vụ đệm/chuyển dữ liệu giữa hai khối.

Phạm vi kiểm chứng hiện tại có giới hạn: chưa kiểm thử FIFO đầy/rỗng kéo dài, wrap-around, overflow/underflow, burst dài, baud lệch, khung sai, Start glitch hoặc các cấu hình parameter khác. Vì vậy, chưa thể kết luận hệ thống chống nghẽn trong mọi điều kiện hoặc khẳng định độ tin cậy tuyệt đối. Các bước kiểm thử tiếp theo nên bổ sung test FIFO biên và burst dài, xác minh trực tiếp Stop bit/framing error, parity và các cờ lỗi FIFO; đồng thời mở rộng giao diện APB hoặc AXI4-Lite nếu cần tích hợp với SoC.

CHƯƠNG 5. **KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN CỦA ĐỀ TÀI**

5.1. Kết luận

Nhìn lại toàn bộ quá trình thực hiện đồ án, tôi đã xây dựng thành công một "trạm trung chuyển" dữ liệu UART hoàn chỉnh từ con số không. Cụ thể, đề tài đã đạt được những kết quả thực tế như sau:

- **Hoàn thiện trạm thu/phát song công:** Hệ thống có khả năng vừa "nghe" vừa "nói" cùng lúc (Full-Duplex) mượt mà, không xảy ra xung đột tín hiệu.
- **Lắp đặt "nhà kho" siêu tốc (FWFT FIFO):** Tích hợp kho đệm thông minh tự động đẩy dữ liệu ra cửa ngay lập tức, triệt tiêu hoàn toàn độ trễ và hiện tượng nghẽn cổ chai.
- **Tự động hóa khâu kiểm tra:** Xây dựng "vị trọng tài" tự động (Python + Testbench) giúp tự động cấp dữ liệu và chấm điểm chính xác 100%, thay thế hoàn toàn việc dò đường sóng thủ công.

5.2. **Những khó khăn trong quá trình thiết kế**

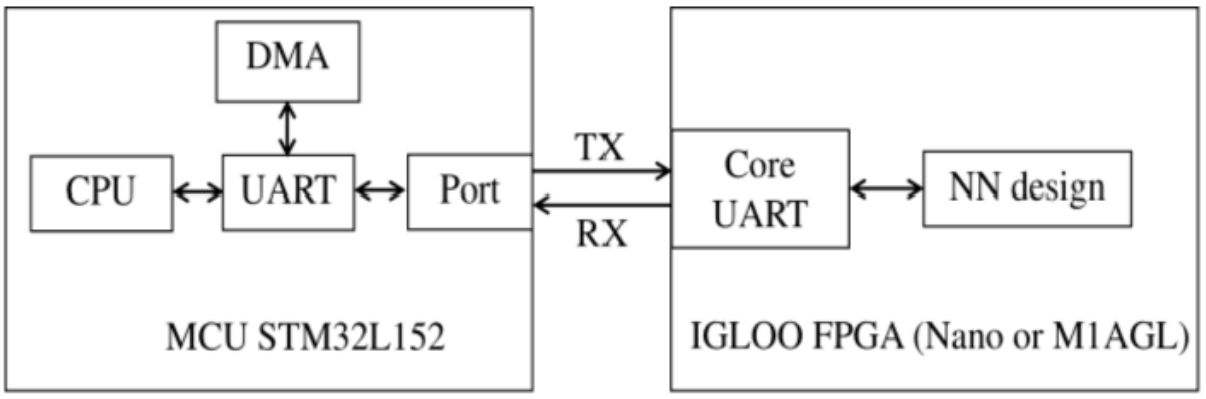
Lỗi đọc lặp dữ liệu (Duplicate Read) do thiếu chốt chặn logic: Ban đầu, tại trạng thái IDLE của khối uart_tx, điều kiện chuyển trạng thái chỉ được đặt là

if(txstart_i). Do máy trạng thái FSM phản ứng quá nhanh so với tốc độ cập nhật con trỏ đọc của FIFO, hệ thống liên tục lấy lại gói dữ liệu cũ (vừa mới truyền xong) để phát thêm lần nữa, dẫn đến kết quả mô phỏng sai lệch. Khó khăn này đã được khắc phục triệt để bằng cách bổ sung thêm cờ ràng buộc !txdone_o vào điều kiện kiểm tra "if(txstart_i && txdone_o)". Chốt chặn này ép khối uart_tx phải đứng đợi 1 nhịp xung nhịp để kho chứa FIFO kịp thời nhích con trỏ, loại bỏ hoàn toàn hiện tượng phát lặp byte.

Khi làm mạch Loopback (mạch nhận dữ liệu từ máy tính rồi lập tức phát ngược lại), tôi đã dùng khối đệm FIFO dạng đồng bộ (Synchronous). Nghĩa là lệnh đọc/ghi phải đợi nhịp clock tiếp theo mới có tác dụng. Hệ quả là bị trễ 2 chu kỳ: byte đầu tiên phát ra là rác (don't care), và khi test trên phần mềm Questa gửi 5 byte (A5, 5A, 31, 32, 33) thì bị kẹt mất 2 byte cuối cùng ngay tại cổng vào. Khắc phục bằng cách khối đệm FIFO từ chế độ "Đọc đồng bộ" sang "Đọc bất đồng bộ". Khi có cờ báo hiệu, dữ liệu nhảy ra ngay lập tức mà không phải chờ xung clock.

5.3. **Hướng phát triển trong tương lai**

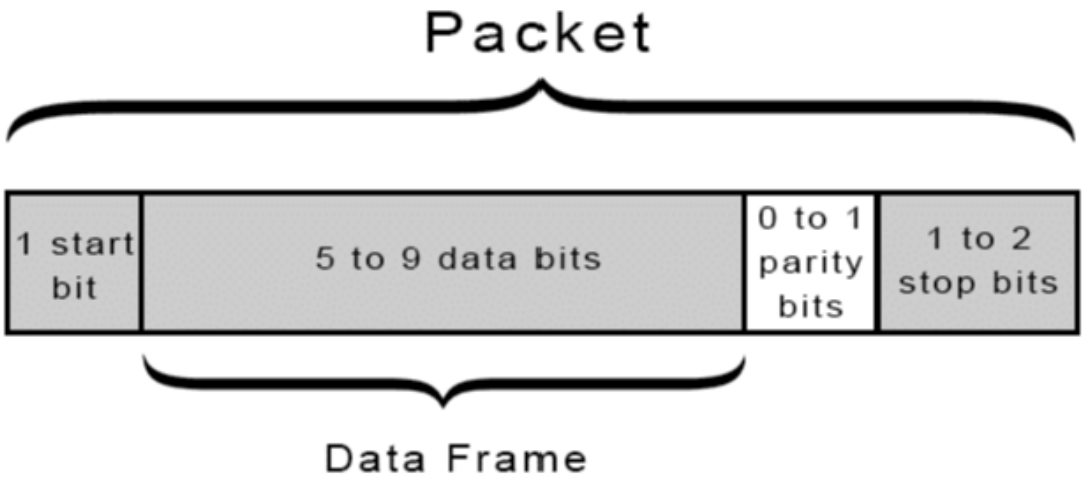
- Dạy UART cách nói chuyện với CPU (Nhúng chuẩn giao tiếp): Hiện tại hệ thống đang đứng độc lập. Hướng đi sắp tới là bọc thêm một lớp giao tiếp chuẩn công nghiệp (như giao thức AMBA APB hoặc AXI). Khi đó, các bộ não CPU hiện đại (như chip RISC-V hay ARM) có thể dễ dàng đọc/ghi dữ liệu vào khối UART này như đọc một thanh ghi bộ nhớ.



Hình 5.1. VD thực tiễn khi UART connect với CPU.

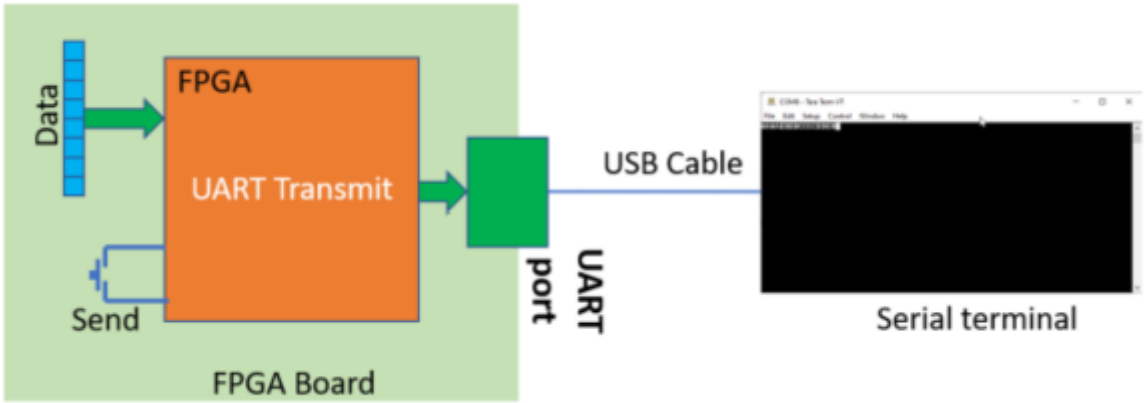
- Thêm tính năng phát hiện "hàng giả, hàng lỗi": Nâng cấp khối nhận (RX) để nó có khả năng tự kiểm tra tính toàn vẹn của gói tin (Parity Check). Nếu phát hiện

gói tin bị móp méo do nhiễu đường truyền, hoặc kho chứa đã đầy mà vẫn bị nhét thêm, nó sẽ bật các cờ báo động đỏ (Error Flags) để hệ thống biết và xử lý.



Hình 5.2. VD về Parity Check trong UART.

- Đưa xuống chạy thử trên "đồ thật": Tiến hành biên dịch đoạn code này để nạp xuống một board mạch FPGA vật lý. Sau đó, dùng dây cáp cắm thẳng từ board mạch vào máy tính (thông qua cổng USB-Serial) để gõ phím và xem hệ thống nhận/phản hồi trực tiếp trong thế giới thực.



Hình 5.3. VD khi UART giao tiếp với FPGA.